

Bachelorarbeit
Studiengang : Informatik

Optimierung des Entwicklungsworkflows in der Webentwicklung durch Codegenerierung und OpenAPI

bei

Heitec AG

von

Nicolas Ostermann

82796

Betreuender Professor: Prof. Dr. Rainer Werthebach

Einreichungsdatum : 31. Mai 2024

Angaben zur Firma

Unternehmen : Heitec AG
Branche : Dienstleistung Software/Automatisierung
Abteilung : Softwareentwicklung
Adresse : Brunnenstraße 36
74654 Crailsheim

Betreuer : Sebastian Holst
Telefon : +49 7951 9366 753
E-Mail : sebastian.holst@heitec.de

Eidesstattliche Erklärung

Hiermit erkläre ich, **Nicolas Ostermann**, dass ich die vorliegenden Angaben in dieser Arbeit bei **Heitec AG** wahrheitsgetreu und selbständig verfasst habe.

Weiterhin versichere ich, keine anderen als die angegebenen Quellen und Hilfsmittel benutzt zu haben, dass alle Ausführungen, die anderen Schriften wörtlich oder sinngemäß entnommen wurden, kenntlich gemacht sind und dass die Arbeit in gleicher oder ähnlicher Fassung noch nicht Bestandteil einer Studien- oder Prüfungsleistung war.

Ort, Datum

Unterschrift (Student)

Kurzfassung

- **Problemstellung:**

Die Entwicklung von Webanwendungen kann oft durch ineffiziente Abläufe und überflüssigen Code erschwert werden, was nicht nur die Produktivität bremst, sondern auch die Fehleranfälligkeit erhöht. Um diese Herausforderungen anzugehen, konzentriert sich diese Arbeit darauf, den Entwicklungsprozess zu optimieren. Dies geschieht durch die Automatisierung häufig wiederkehrender Aufgaben und die Standardisierung der Codebasis mit Hilfe von Codegenerierung und OpenAPI.

- **Methodik:**

Für die Bewältigung dieser Herausforderungen wurde ein verbesserter Workflow entwickelt, der Codegenerierungstools und OpenAPI-Spezifikationen integriert. Der Prozess begann mit einer Analyse der aktuellen Entwicklungspraktiken, um Schwachstellen zu identifizieren, gefolgt von der Entwicklung eines Prototyp-Tools zur Automatisierung der Generierung von Client- und Servercode. Die Methodik umfasste umfangreiche Literaturstudien, Interviews mit Entwicklern und die Analyse von Daten aus bestehenden Projekten.

- **Erzielter Fortschritt:**

Durch den entwickelten Ansatz konnte der manuelle Programmieraufwand erheblich reduziert werden, was zu einer verbesserten Konsistenz und Wiederverwendbarkeit des Codes führte. Während einer Testphase in einem test Projektumfeld wurde eine signifikante Beschleunigung der Entwicklungsprozesse und eine Reduktion der Fehlerquote festgestellt. Diese Ergebnisse bestätigen, dass die Verwendung von automatisierter Codegenerierung und OpenAPI-Spezifikationen eine effektive Methode ist, um die Effizienz und Qualität in der Webentwicklung zu steigern.

- **Schlussfolgerung:**

Diese Arbeit zeigt, dass die Einführung automatisierter Prozesse im Entwicklungsablauf, unterstützt durch Technologien wie OpenAPI, zu deutlichen Effizienzsteigerungen und Qualitätsverbesserungen führen kann. Unternehmen, die moderne Webanwendungen entwickeln, können durch die Übernahme dieses Ansatzes ihren Workflow optimieren.

Inhaltsverzeichnis

Angaben zur Firma	i
Eidesstattliche Erklärung	ii
Kurzfassung	iii
Inhaltsverzeichnis	iv
Abbildungsverzeichnis	ix
Tabellenverzeichnis	x
Quelltextverzeichnis	xi
Abkürzungsverzeichnis	xii
1. Einleitung	1
1.1. Vorstellung des Unternehmens	1
1.2. Motivation	2
1.3. Problemstellung und -abgrenzung	3
1.4. Ziel der Arbeit	3
1.5. Vorgehen	4

2. Grundlagen	7
2.1. Angular	7
2.1.1. Grund Konzepte	7
2.1.2. Erweiterte Konzepte	8
2.2. .NET	8
2.2.1. Grund Konzepte	8
2.2.2. Erweiterte Konzepte	9
2.3. TypeScript	9
2.4. Node.js	10
2.5. NPM	11
2.6. OpenAPI	12
2.7. Agile	13
2.7.1. Vier Grundpfeiler des Agilen Manifests	13
2.7.2. Kanban	14
2.8. Wasserfall Modell	15
3. Problemanalyse	16
3.1. Vorgehen der Analyse	16
3.1.1. Datenrecherche	16
3.1.2. Strukturiertes Interview	17
3.1.3. Fallstudienanalyse	17
3.2. Analyse	18
3.2.1. Ausgewertete Daten und Statistiken	18
3.2.2. Interview mit Tech Lead Paul Schnabel	27
3.2.3. Auswertung Fallstudien	29
3.3. Ergebnis der Analyse	37

4. Lösungskonzept	38
4.1. Projektkontext und Notwendigkeit der Codegenerierung	39
4.2. Rolle der Teamkommunikation	40
4.2.1. Zeit- und Kostenersparnis durch Codegenerierung	40
4.3. Vorteile der Codegenerierung spezifisch für Angular	42
4.3.1. Auswahl von Angular	42
4.3.2. Best Practices, Clean Code und einheitlicher Coding-Stil	43
4.4. Konzept basierend auf Checkliste	44
4.4.1. ROI-Analyse	44
4.4.2. Modeling and Problem Formulation	44
4.4.3. Solution Design	46
4.4.4. Data Collection	48
4.4.5. Optimization	49
4.4.6. Validation	51
5. Implementierung	52
5.1. Entwicklungsansatz	52
5.1.1. Kernfunktionen	52
5.1.2. Modularität und Separation of Concerns	54
5.1.3. Dokumentation und Code-Style	55
5.2. Generierung des Angular-Codes	55
5.2.1. Modularität durch Serviceorientierte Architektur	56
5.2.2. Code-Struktur und -Qualität	57
5.3. Implementierung des CLI-Tools	58
6. Inbetriebnahme	60
6.1. Konfiguration der <code>package.json</code> und <code>tsconfig.json</code>	60

6.2. Erstellung der <code>.npmrc</code> Datei	61
6.3. Build und Deployment	61
7. Evaluierung	62
7.1. Einführung	62
7.2. Ziel 1: Definition und Implementierung des Entwicklungsworkflows .	62
7.2.1. Erwartetes Ergebnis	62
7.2.2. Erreichte Ergebnisse	62
7.2.3. Diskrepanzen und deren Ursachen	62
7.3. Ziel 2: Reduktion von Boilerplate-Code	63
7.3.1. Erwartetes Ergebnis	63
7.3.2. Erreichte Ergebnisse	63
7.3.3. Diskrepanzen und deren Ursachen	63
7.4. Ziel 3: Steigerung der Effizienz, Qualität und Wartbarkeit der Softwa- reentwicklung	63
7.4.1. Erwartetes Ergebnis	63
7.4.2. Erreichte Ergebnisse	63
7.4.3. Diskrepanzen und deren Ursachen	63
7.5. Ziel 4: Sicherstellung der Erweiterbarkeit	64
7.5.1. Erwartetes Ergebnis	64
7.5.2. Erreichte Ergebnisse	64
7.5.3. Diskrepanzen und deren Ursachen	64
7.6. Vergleich mit dem alten Codegenerierungstool	64
7.7. Umfrage	65
7.7.1. Kontext der Umfrage	65
7.7.2. Bewertungsübersicht	65
7.7.3. Spezifisches Feedback	66

7.8. Abschließende Gedanken	66
7.9. Evaluation des theoretischen Ansatzes	67
7.9.1. Methodischer Ansatz	67
7.9.2. Evaluation der verwendeten Quellen	67
8. Zusammenfassung und Ausblick	68
8.1. Erreichte Ergebnisse	68
8.2. Ausblick	68
8.2.1. Erweiterbarkeit der Ergebnisse	68
8.2.2. Übertragbarkeit der Ergebnisse	69
Literatur	70
A. Anhang Interview	74
B. Anhang Code	78
B.1. Vergleich Dateiaufbau	78
B.2. Vergleich ErrorHandling	80
C. Anhang Umfrage	81
C.1. Umfragebogen	81
C.2. Fragen	82

Abbildungsverzeichnis

1.1. Workflow Diagramm	5
3.1. Performance von unterschiedlichen IT-Projekten[30]	19
3.2. Erfolgsrate von Projekten in Prozent[19]	20
3.3. Ranking Sources of Technical Debt [15]	23
3.4. Participant's perceptions regarding how they deal with Clean Code. [25]	25
4.1. Web Framework Popularity Over Time [42]	42
4.2. Use case diagram	45
4.3. Sequence diagram	47
4.4. Vereinfachter Workflow mit Client Generierungstool	50
5.1. CLI-Tool	59

Tabellenverzeichnis

3.1. Softfaktoren und ihre Auswirkungen auf die Produktivität [48]	30
3.2. Technische Faktoren und ihre Auswirkungen auf die Produktivität [48]	31
3.3. Kommunikationsprobleme in agilen Teams [50]	36

Listings

5.1. Generierung von Angular Services	54
5.2. Funktion mit Sideeffect	55
5.3. Erstellung des Konstruktors	56
5.4. Implementierung des spinners	58
5.5. Rückmeldung für den User	58
B.1. Aufbau einer Datei im bekannten CodeGeneration Tool	78
B.2. Aufbau einer Datei im neuen CodeGeneration Tool	79
B.3. Errorhandling im bekannten CodeGeneration Tool	80
B.4. Errorhandling im neuen CodeGeneration Tool	80

Abkürzungsverzeichnis

UI	User Interface	3
LLM	Large Language Models	3
REST	Representational State Transfer	3
API	Application Programming Interface	3
KI	Künstliche Intelligenz	3
SPA	Single Page Applications	7
HTML	Hypertext Markup Language	7
DOM	Document Object Model	7
RxJS	Reactive Extensions for JavaScript	8
NgRx	Angular libraries for Reactive extensions	8
MVC	Model View Controller	8
CLR	Common Language Runtime	9
ECMA	European Computer Manufacturers Association	10
NPM	Node Package Manager	11
JSON	JavaScript Object Notation	12
HTTP	Hypertext Transfer Protocol	12
YAML	YAML Ain't Markup Language	12
CIO	Chief Information Officer	21
CRM	Customer Relationship Management	21
ERP	Enterprise Resource Planning	21
ROI	Return on Investment	33
CMMI	Capability Maturity Model Integration	35
MVP	Minimum Viable Product	46
ISP	Interface Segregation Principle	56
CLI	Command Line Interface	58
SPS	Speicherprogrammierbare Steuerung	68

1. Einleitung

Ein wichtiger Bestandteil der digitalen Transformation, die Unternehmen aller Größen und Branchen durchlaufen, ist die Entwicklung von Webanwendungen. Der Erfolg softwarebezogener Projekte hängt entscheidend von Faktoren wie Effizienz, Anpassungsfähigkeit und Geschwindigkeit in diesem sich schnell verändernden Umfeld ab. Trotz bemerkenswerter Fortschritte bei den Entwicklungstechniken und den verfügbaren Technologien sehen sich die Entwicklungsteams mit einer Reihe immer komplexer werdenden Herausforderungen konfrontiert. Dazu gehören die Verwaltung komplexer Anforderungen und die Integration neuer technologischer Lösungen in bestehende Systemarchitekturen, um die sich ändernden Bedürfnisse der Kunden zu erfüllen.

1.1. Vorstellung des Unternehmens

Im Jahr 1973 legte Richard Heindl den Grundstein für das heutige Technologieunternehmen, indem er eine Beratungs- und Konstruktionsfirma im Bereich Maschinen- und Anlagenbau gründete. Der Umzug des Unternehmenssitzes von Nürnberg nach Erlangen im Jahr 1984 ebnete den Weg für die spätere Gründung der HEITEC GmbH.

Zu Beginn zählte das Unternehmen neben Heindl drei Mitarbeiter. Im Laufe der Jahre expandierte HEITEC erheblich. Der erste Schritt dieser Expansion war die Eröffnung einer Niederlassung in Crailsheim im Jahr 1989. Es folgten weitere Ausbauaktivitäten in Deutschland, die zwischen 1991 und 1999 zur Gründung von Niederlassungen in Chemnitz, Regensburg, Neckarsulm-Heilbronn sowie neuen Standorten in München, Augsburg, Kiel, Hamburg und Bönningheim führten. Darüber hinaus baute HEITEC seine internationale Präsenz durch die Gründung von Gesellschaften und Beteiligungen im Ausland aus.

In den 2000er Jahren vollzog sich ein Teil der Unternehmensgruppe in eine Aktiengesellschaft. Zur Feier ihres Firmenjubiläums bezog HEITEC 2004 ein neues Hauptquartier in der Güterbahnhofstraße 5 in Erlangen, das seit dem 1. Februar 2017 offiziell der Hauptsitz des Unternehmens ist.

Über die Jahre hat sich HEITEC stetig weiterentwickelt und seine Präsenz durch die Eröffnung eines neuen Standortes in Eckental erweitert. Mit mehr als 1.200 Beschäftigten und einem Umsatz von über 140 Millionen Euro im Jahr 2022 stellt HEITEC seine hohe Qualifikation und Expertise unter Beweis. Rund 60 % der Belegschaft besteht aus Hochschulabsolventen und Technikern. Das Unternehmen bietet umfassende Industrielösungen für mehr als 2.000 Kunden aus dem Mittelstand und der Großindustrie.

Die HEITEC AG gliedert sich in sechs Hauptabteilungen: Automatisierung, Produktionssysteme, Mess- und Prüftechnik, Elektronik, Softwarelösungen sowie Validierung und Dokumentation. Am Standort Crailsheim liegt der Fokus auf den Bereichen Automatisierung, Softwareentwicklung sowie Validierung und Dokumentation. [20]

1.2. Motivation

In diesem Rahmen spielt die Optimierung des Entwicklungsflusses eine zentrale Rolle, um sowohl die Produktivität zu steigern als auch die Qualität der Softwareprodukte zu verbessern. Dies gilt insbesondere für die Disziplin der Webentwicklung, in der Fullstack-Entwickler mit diversen Frontend- und Backend-Technologien arbeiten. Angesichts dieser Vielfalt wird es immer schwieriger, die Entwicklungsprozesse zu koordinieren.

Die Entwicklung von Webanwendungen ist ein wichtiger Bestandteil der digitalen Transformation von Unternehmen. Dieser Prozess erfordert nicht nur technisches Know-how und Ressourcen, sondern auch eine ständige Anpassung an neue Entwicklungen und Marktanforderungen. Die größte Herausforderung besteht darin, den Softwareentwicklungsprozess effektiver zu gestalten, ohne dabei die Qualität oder Anpassungsfähigkeit der Software zu beeinträchtigen. Vor diesem Hintergrund gewinnt die Optimierung von Entwicklungsabläufen an Bedeutung. Sie ermöglicht es, schneller auf Veränderungen zu reagieren, die Produktivität zu steigern und letztlich wettbewerbsfähige Softwarelösungen bereitzustellen.

Bei der Webentwicklung ist die Vielfalt der zu integrierenden Technologien Chance und Herausforderung zugleich. Die Aufgabe von Full-Stack-Entwicklern ist es, diese Vielfalt an Technologien und Frameworks zu beherrschen und zu verbinden. Diese Komplexität erfordert eine genaue Koordinierung die häufig das Erstellen und Warten von umfangreichem Boilerplate-Code¹ bedingt. Diese Aufgaben sind nicht nur zeitaufwendig, sondern bergen auch das Risiko von Fehlern und Inkonsistenzen, die die Gesamtqualität der Projekte beeinträchtigen können.

¹"In der Informationstechnologie ist ein Boilerplate ein Code-Segment, das ohne Änderung immer wieder verwendet werden kann." [8]

1.3. Problemstellung und -abgrenzung

Wenn es um die Optimierung von Entwicklungsprozessen geht, steht die Webentwicklung vor Hürden, die mit den Bedürfnissen der Kunden zusammenhängen. Die derzeitigen Entwicklungspraktiken, insbesondere bei Projekten, die Full-Stack-Entwicklungsansätze verwenden, leiden unter Ineffizienzen, die durch sich wiederholende und fehleranfällige Aufgaben verursacht werden.

Die Erstellung und Pflege von Boilerplate-Code, der für die Einrichtung und den Betrieb von Standardsoftwarekomponenten erforderlich ist, ist ein sehr zeitaufwändiger Teil des Entwicklungsprozesses. Dies verlangsamt nicht nur die Entwicklung, sondern beeinträchtigt auch die Einheitlichkeit und Klarheit des Codes, was sich auf die Codequalität, Wartbarkeit und Skalierbarkeit der Software auswirkt. Daher ist die Durchsetzung von Best Practices zur Sicherstellung der Erstellung eines einheitlichen Codes von großer Bedeutung für die Erhaltung der Softwarequalität und Erweiterbarkeit.

In dieser Arbeit wird der Fokus auf die Optimierung des Entwicklungsworkflows durch Codegenerierung² im Rahmen von Representational State Transfer (REST)-Application Programming Interface (API)s gelegt. Es werden weder Künstliche Intelligenz (KI)-Tools, Large Language Models (LLM) oder Natural language processing noch die Generierung von allgemeinen User Interface (UI)-Komponenten berücksichtigt. Das Hauptmerkmal dieser Arbeit ist die Automatisierung der Verbindung zwischen den Client- und Serverkomponenten.

1.4. Ziel der Arbeit

Das übergeordnete Ziel dieser Arbeit ist die Optimierung des Arbeitsablaufs zwischen Client- und Server-Verbindungen durch die Schaffung eines neuen, definierten Workflows für die Client-Erstellung. Dies soll dadurch erreicht werden, dass der Umfang der manuellen Erstellung von Boilerplate-Code reduziert wird. Es ist zu erwarten, dass die Implementierung dieser neuen Arbeitsabläufe die Qualität, Haltbarkeit und Effizienz der Softwareprodukte erheblich verbessern wird. Dabei ist es von entscheidender Bedeutung, dass die Erweiterbarkeit der entwickelten Systeme gewährleistet bleibt. Konkret sollen die folgenden Ziele erreicht werden:

²Codegenerierung bezieht sich auf den Prozess der automatischen Generierung von Code auf der Grundlage einer Beschreibung auf höherer Ebene.

- **Definition und Implementierung eines optimierten Workflows:**
Ein optimierter Workflow soll einen klaren Prozess schaffen, der den Informationsaustausch zwischen Client- und Serveranwendungen effizienter macht. Dazu gehören die Automatisierung sich wiederholender Aufgaben und die Verringerung der Notwendigkeit menschlicher Eingriffe während des Kodierungsprozesses.
- **Reduktion von Boilerplate-Code:**
Der Aufwand für die Erstellung von standardisiertem Code, der in vielen Projekten ähnlich ist, soll durch den Einsatz von Vorlagen und Automatisierungstools minimiert werden. Dies trägt dazu bei, den Entwicklungsprozess zu beschleunigen und die Fehlerquellen zu verringern.
- **Steigerung der Effizienz, Qualität und Wartbarkeit der Softwareentwicklung:**
Durch Straffung von Arbeitsabläufen und Reduzierung von "Boilerplate"Code soll die Softwareentwicklung effizienter und qualitativ besser werden. Zusätzlich wird versucht, die Wartbarkeit und Zukunftssicherheit der Softwareprodukte zu erhöhen, um langfristig Zeit und Ressourcen zu sparen.
- **Sicherstellung der Erweiterbarkeit:**
Auch wenn der Entwicklungsprozess optimiert und automatisiert ist, ist es entscheidend, dass die entwickelten Systeme flexibel bleiben und leicht an neue Anforderungen angepasst werden können. Eine Schlüsselkomponente des neuen Arbeitsablaufs sollte daher die Erweiterbarkeit der Softwareprodukte sein.

Das Erreichen dieser Ziele wird als Maßstab für die Bewertung der Wirksamkeit dieser Arbeit herangezogen. Es wird erwartet, dass die Umsetzung der definierten Arbeitsabläufe nicht nur die Entwicklungsprozesse im Projektkontext verbessern, sondern auch einen Beitrag zur allgemeinen Praxis der Softwareentwicklung leisten werden.

1.5. Vorgehen

In dieser Bachelorarbeit wird ein systematischer Ansatz verfolgt, der sich über mehrere Phasen erstreckt, um eine umfassende Optimierung der Entwicklungsabläufe in der Webentwicklung zu erreichen.

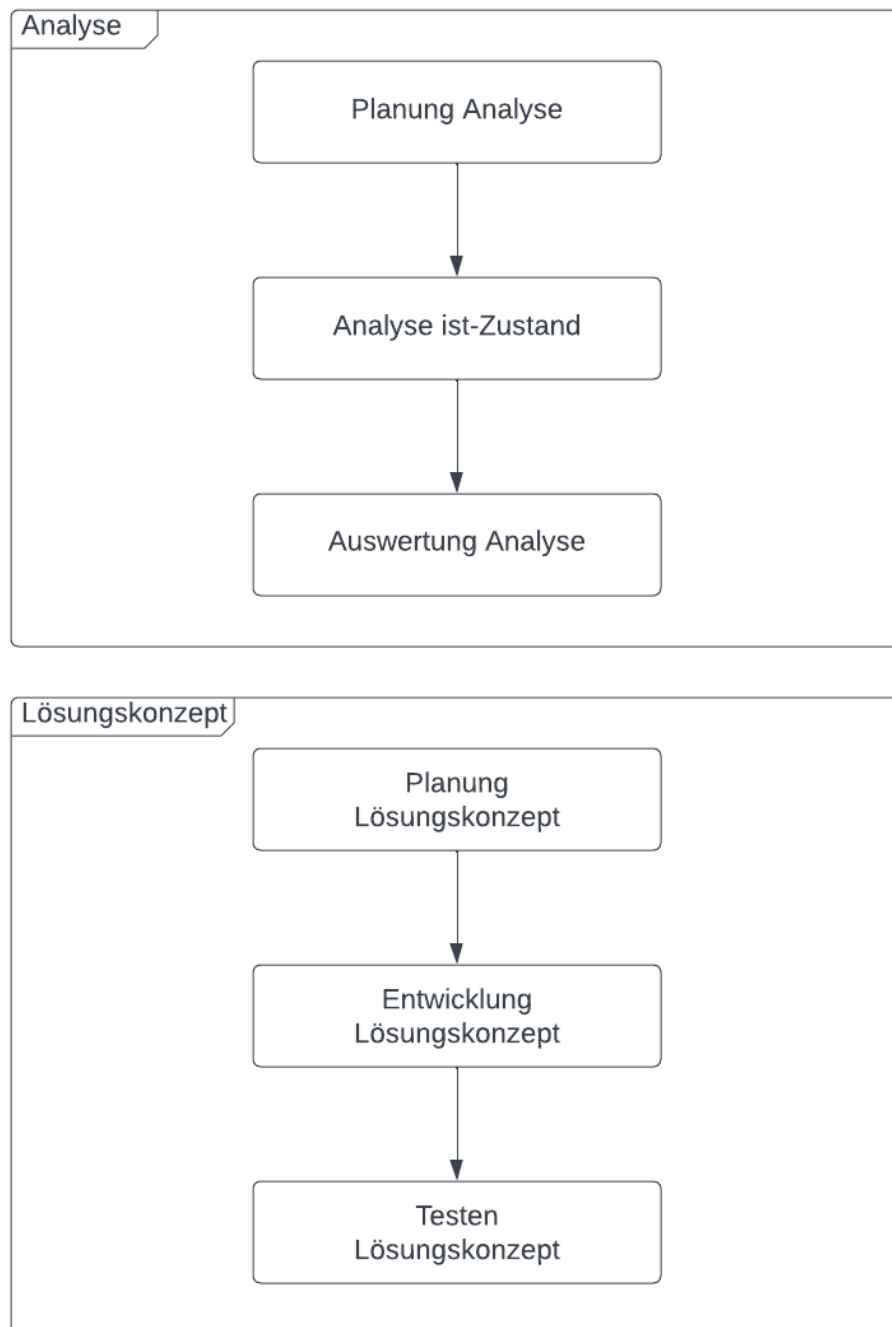


Abbildung 1.1.: Workflow Diagramm

- **Planung der Analyse:**
In dieser Phase wird der Projektrahmen definiert. Der Untersuchungsgegenstand und die Ziele werden festgelegt. Es wird ein Plan erstellt, der alle folgenden Schritte abdeckt.
- **Analyse des Ist-Zustands:**
Hier wird die aktuelle Situation analysiert, um ein klares Verständnis der vorhandenen Prozesse, Probleme und Möglichkeiten zu erlangen. Diese Phase umfasst die Sammlung und Analyse von Informationen über die aktuellen Arbeitsabläufe.
- **Auswertung der Analyseergebnisse:**
Nach der Untersuchung werden die Ergebnisse analysiert. Dabei werden die Hauptprobleme identifiziert und der Soll-Zustand definiert. Die Entwicklung des Lösungskonzepts baut auf dieser Bewertung auf.
- **Planung des Lösungskonzepts:**
In der Planungsphase des Lösungskonzepts werden zunächst die am besten geeigneten Technologien und Methoden für die Entwicklung und Umsetzung des Konzepts ausgewählt. Danach werden die Ziele des Projekts ausführlich beschrieben, um einen klaren Rahmen für die folgenden Schritte zu schaffen. Daraufhin wird ein detaillierter Entwicklungs- und Implementierungsplan erstellt, der sowohl technische Spezifikationen als auch methodische Ansätze berücksichtigt.
- **Entwicklung des Lösungskonzepts:**
Das Lösungskonzept wird nach gründlicher Vorplanung konzipiert und umgesetzt. Während der Entwicklungsphase werden Konzepte kontinuierlich angepasst und optimiert, um ihre Effektivität zu maximieren.
- **Testen des Lösungskonzepts:**
Das Konzept der Lösung wurde getestet. In dieser Phase werden die vorgeschlagenen Lösungen überprüft und gegebenenfalls Anpassungen vorgenommen.

Diese Methodik sichert ein fokussiertes Herangehen zur Verbesserung des Entwicklungsprozesses, indem jeder Schritt auf dem vorangegangenen aufbaut und so zu einer effizienten und wirkungsvollen Lösung beiträgt.

2. Grundlagen

2.1. Angular

Angular, ein Open-Source-Framework für Webanwendungen von Google, hat sich seit seiner Einführung als AngularJS im Jahr 2010 fest etabliert. Über die Jahre hinweg hat Angular große Veränderungen durchlaufen. Die weitreichendste dieser Veränderungen war die umfassende Neugestaltung im Jahr 2016, welche die Evolution von AngularJS zu dem heute als Angular 2+ oder schlicht als Angular bekannten Framework markierte. Mit Angular 2+ kam auch der Umstieg von JavaScript zu TypeScript (2.3). [18][39][43]

Angular verfolgt dabei das Ziel, Single Page Applications (SPA) durch effiziente Updates des Document Object Model (DOM) zu unterstützen, Interaktionen zu vereinfachen und skalierbare Strukturen zu schaffen. Es nutzt Hypertext Markup Language (HTML) als Vorlagensprache, und erweitert dessen Syntax, um Anwendungen mit klar definierten Komponenten zu erstellen.[18]

2.1.1. Grund Konzepte

1. **Components und Data Binding:** Ermöglichen eine deutliche Abgrenzung zwischen dem Visuellen und dem Logiklayer, was das automatische Aktualisieren der Ansicht bei Modifikationen am Modell vereinfacht.[43]
2. **Services und Dependency Injection:** Services bieten wiederverwendbaren Code, während Dependency injection die Verknüpfung zwischen Komponenten minimiert und die Testbarkeit verbessert.[43]
3. **Module und Modularität:** Angular strukturiert den Code in Modulen, wodurch das Verwalten umfangreicher Anwendungen erleichtert und die Wiederverwendung von Code unterstützt wird.[43]
4. **Routing und Navigation:** Der Router von Angular erleichtert in SPA die Navigation, indem URLs auf Komponenten abgebildet werden, was ein flüssiges Nutzererlebnis ohne das Neuladen von Seiten ermöglicht.[43]

2.1.2. Erweiterte Konzepte

1. **RxJS:** Reactive Extensions for JavaScript (RxJS) unterstützt eine effiziente, deklarative Verarbeitung von Datenströmen und die Behandlung asynchroner Ereignisse.[43]
2. **NgRx:** Angular libraries for Reactive extensions (NgRx) bietet ein ausgereiftes Muster zur Verwaltung des Zustands einer Anwendung.[43]
3. **Testing und Testautomatisierung:** Für die Qualitätssicherung stehen Werkzeuge wie Jasmine in Kombination mit den Angular Testing Utilities bereit.[43]

Abschließend sei hervorgehoben, dass Angular oft gemeinsam mit leistungsfähigen Backend-Technologien wie .NET (2.2) eingesetzt wird, was seinen starken Einsatz in Unternehmensumgebungen mit anspruchsvollen Geschäftsanforderungen begründet. Diese Kombination begünstigt die Schaffung von Full-Stack-Lösungen, die durch hohe Leistungsfähigkeit und Skalierbarkeit charakterisiert sind.

2.2. .NET

.NET ist eine Entwicklungsplattform von Microsoft, die Entwicklern Werkzeuge für die moderne Webentwicklung bereitstellt. Die Stärke von .NET liegt in seiner Fähigkeit, Anwendungen zu unterstützen, die nahtlos auf unterschiedlichen Geräten und Betriebssystemen laufen.[13] Nach der offiziellen Einführung im Jahr 2002 wurde .NET immer wieder durch neue Funktionen und Programmiermodelle erweitert. Ein Meilenstein war die Einführung von .NET Core im Jahr 2016 – einer plattformübergreifenden, leistungsfähigeren Version von .NET.[2] .NET unterstützt verschiedene Programmiersprachen wie C#, VB, .NET und F#, womit Entwickler die Freiheit haben, Web-, Desktop-, Mobil- und Cloud-Anwendungen zu erstellen.[14]

2.2.1. Grund Konzepte

1. **Model View Controller (MVC):** Der MVC fördert eine klare Trennung zwischen der Benutzeroberfläche (View), der Geschäftslogik (Controller) und der Datenmodellschicht (Model). Dies unterstützt die Wartung und Skalierbarkeit von Anwendungen.[26]
2. **Standard Library:** Die Standard library sichert die nahtlose Zusammenarbeit verschiedener .NET-Implementierungen und bietet dadurch Entwicklern eine durchgehend konsistente Erfahrung.[26]

3. **Common Language Runtime (CLR):** CLR ist für die Ausführung von .NET-Anwendungen verantwortlich und ermöglicht Features wie Speicherverwaltung, Ausnahmebehandlung und Sicherheit.[26]

2.2.2. Erweiterte Konzepte

1. **Asynchronität:** Das async/await-Muster in der asynchronen Programmierung erleichtert die Gestaltung von Operationen, die das Blockieren von Prozessen vermeiden.[26]
2. **Microservices:** Durch .NET Core gefördert, ermöglicht die Microservices-Architektur die Entwicklung von skalierbaren und anpassungsfähigen Anwendungssystemen.[26]

Zusammengefasst stellt .NET eine robuste und vielseitige Plattform dar, die für die Entwicklung von Anwendungen über ein breites Spektrum von Domänen hinweg konzipiert ist.

2.3. TypeScript

Als ein von Microsoft kreierte und als Open-Source zugängliches Projekt, dient TypeScript als eine Erweiterung von JavaScript. Dies impliziert, dass sämtlicher JavaScript-Code ohne Weiteres auch als TypeScript-Code gültig ist. Durch die Integration von Funktionen wie statischer Typisierung sowie der Einführung von Klassen und Interfaces ergänzt TypeScript JavaScript.[45]

Das Hauptziel von TypeScript liegt in der Vereinfachung und Optimierung der Entwicklung umfangreicher Softwareprojekte. Es stellt eine solide Codegrundlage zur Verfügung und unterstützt Entwickler durch Werkzeuge, die bereits während der Kompilierungsphase Fehler identifizieren können. Durch die Einführung einer Typüberprüfung zur Kompilierungszeit trägt TypeScript dazu bei, potenzielle Fehlerquellen frühzeitig zu identifizieren und somit zu vermeiden.[45]

1. **Statische Typisierung:** Bietet Entwicklern die Möglichkeit, Variablen, Funktionsparametern sowie Rückgabetypen spezifische Typen zuzuordnen, was nicht nur die Codequalität steigert, sondern auch eine frühzeitige Fehlererkennung während der Kompilierung ermöglicht.[31]
2. **Klassen und Interfaces:** Ermöglicht die Nutzung moderner objektorientierter Programmierprinzipien, die bereits mit JavaScript ES6 eingeführt wurden, ergänzt durch weitere Elemente wie Interfaces. Diese fördern eine strenge Typüberprüfung und die Definition von Verträgen zwischen verschiedenen Teilen eines Projekts.[31]
3. **European Computer Manufacturers Association (ECMA) Script Standards:** TypeScript hat eine enge Verbindung zu den ECMAScript-Standards und fördert die Anwendung aktuellster JavaScript-Funktionen, was die zukunftssichere Entwicklung von Projekten unterstützt. Die vollständige Kompatibilität mit ECMAScript-Standards erlaubt es Entwicklern, moderne JavaScript-Funktionen zu nutzen und dabei von den zusätzlichen Typisierungs- und Programmierfunktionen von TypeScript zu profitieren. Da TypeScript in JavaScript kompiliert wird, das den ECMAScript-Standards entspricht, kann TypeScript-Code in jeder JavaScript-fähigen Umgebung ausgeführt werden.[12][31]

2.4. Node.js

Im Jahr 2009 entwickelte Ryan Dahl Node.js um JavaScript - eine bis dahin im Browser genutzte Programmiersprache - in serverseitigen Anwendungen zu nutzen. Die Motivation für dieses Vorhaben zog Dahl aus den Beschränkungen, die ihm bei der Nutzung traditioneller Webserver aufgefallen waren, vor allem deren mangelnde Fähigkeit, zahlreiche simultane Verbindungen effizient zu verwalten. Diese Analyse führte zur Entwicklung von Node.js, das ein nicht-blockierendes Input/Output-Modell enthält, das die Erstellung von skalierbaren Netzwerk-Apps vereinfacht und optimiert.[41]

Key-Features:

1. **Non-Blocking I/O Modell:** Operationen, die nicht in Node.js stattfinden, blockieren die Anwendung nicht.[41]
2. **Single-Threaded mit Asynchronous Operations:** Node.js arbeitet mit einem einzigen Thread. Trotzdem nutzt es asynchrone Operationen effektiv. Dadurch kann es eine hohe Gleichzeitigkeit unterstützen und Tausende von gleichzeitigen Verbindungen auf einer einzigen Serverinstanz verwalten.[41]

3. **V8 JavaScript Engine:** Node.js baut auf der V8 Engine von Google auf, dadurch profitiert man von stetigen Verbesserungen.[41]
4. **NPM (2.5) Ecosystem:** Das Node Package Manager (NPM) Ökosystem ist eine der größten Open-Source-Bibliotheken weltweit. NPM vereinfacht den Prozess der gemeinsamen Nutzung und Wiederverwendung von Code (siehe 2.5).[41]

Node.js zeichnet sich durch seine Anpassungsfähigkeit aus, die eine Vielfalt von Projekten unterstützt - von simplen Werkzeugen bis hin zu komplexen Web-Services und -Anwendungen. Insbesondere profitieren Echtzeitapplikationen, Microservices und APIs von seiner nicht-blockierenden Struktur, was Node.js zu einer Lösung für ein breites Spektrum an Projekten macht.

2.5. NPM

NPM stellt ein unverzichtbares Instrument im zeitgenössischen Softwareentwicklungsprozess dar, insbesondere wenn es um die Erstellung von Anwendungen mit JavaScript und Node.js geht. Als Paketverwaltungssystem bietet NPM Entwicklern die Möglichkeit, Codebibliotheken zu publizieren, zu finden, zu implementieren und effizient zu verwalten. [1]

Funktionen:

1. **Paketverwaltung:** NPM erleichtert das Hinzufügen, Aktualisieren und Entfernen von Codepaketen in Projekten, wodurch die Abhängigkeitsverwaltung automatisiert wird.[1]
2. **Ökosystem:** Mit über einer Million Paketen bildet NPM das größte Software-Register der Welt, das eine breite Palette von Lösungen für diverse Entwicklungsbedürfnisse bietet.[1]
3. **Versionierung und Kompatibilität:** NPM unterstützt Semantische Versionierung¹, ein Schema, das hilft, Kompatibilitätsprobleme zwischen verschiedenen Paketversionen zu vermeiden.[1]
4. **Skriptausführung:** Entwickler können NPM-Skripte definieren, die häufige Aufgaben wie Tests, Builds und Deployment automatisieren.[1]

¹Versionierungsschema für klare Kommunikation von Softwareänderungen[1]

Komponenten:

1. **NPM-CLI:** Eine Befehlszeilenschnittstelle, die die Interaktion mit NPM ermöglicht.[\[1\]](#)
2. **NPM-Registry:** Eine öffentliche Datenbank, die Informationen über Pakete, ihre Versionen und Abhängigkeiten enthält.[\[1\]](#)
3. **package.json:** Eine Datei in Node.js (2.4)-Projekten, die Projektabhängigkeiten, Skripte und andere Konfigurationen spezifiziert.[\[1\]](#)

NPM ist nicht nur ein Werkzeug zur Packageverwaltung, sondern fördert die Zusammenarbeit und Wiederverwendung von Code im Javascript-Ökosystem.

2.6. OpenAPI

OpenAPI ist eine Spezifikation, die eine standardisierte und sprachunabhängige Schnittstelle für Hypertext Transfer Protocol (HTTP)-basierte APIs definiert. Sie ermöglicht es sowohl Menschen als auch Computern, die Fähigkeiten eines Dienstes zu erkennen und zu verstehen, ohne Zugang zum Quellcode, zur Dokumentation oder zur Netzwerkverkehrsanalyse zu benötigen. Sie bietet eine klare Beschreibung der API, einschließlich der verfügbaren Operationen, benötigten Parameter und erwarteten Antworten. Damit erleichtert OpenAPI die Entwicklung von kompatiblen Client- und Serveranwendungen. Die Spezifikation unterstützt sowohl JavaScript Object Notation (JSON) als auch YAML Ain't Markup Language (YAML) und ermöglicht die Generierung von Dokumentation, Client- und Server-Code sowie Testwerkzeugen, was sie zu einem wertvollen Werkzeug für Entwickler macht. OpenAPI fördert somit eine effiziente und fehlerfreie API-Integration und -Entwicklung. [\[40\]](#)

2.7. Agile

Agile Methodiken repräsentieren einen Wandel in der Praxis der Softwareentwicklung, indem sie von konventionellen, linearen Prozessen abrücken und sich einem anpassungsfähigen, zyklischen Vorgehen zuwenden. Dieses Vorgehen zeichnet sich besonders durch seine Flexibilität in sich schnell verändernden Entwicklungsumgebungen aus. Die philosophischen Grundpfeiler dieser Methoden wurden im Agile Manifest von 2001 festgehalten. Das agile Manifest ist das Fundament für agile Praktiken wie Scrum, Kanban (2.7.2) und Extreme Programming. Angesichts der rasch fortschreitenden technologischen Evolution und den wachsenden Bedürfnissen nach Flexibilität sowie schneller Anpassungsfähigkeit an Marktveränderungen, haben sich diese Praktiken durchgesetzt.[32]

2.7.1. Vier Grundpfeiler des Agilen Manifests

1. **Individuen und Interaktionen über Prozesse und Werkzeuge:** Agile Modelle priorisieren direkte Kommunikation und Teaminteraktionen, um effektive Lösungsstrategien zu entwickeln und umzusetzen.[32][22]
2. **Funktionierende Software über umfassende Dokumentation:** Der Fokus liegt auf der regelmäßigen Auslieferung nutzbarer Softwareversionen, die den Kernanforderungen der Kunden entsprechen.[32][22]
3. **Kundenkollaboration über Vertragsverhandlung:** Enger und kontinuierlicher Austausch mit Kunden wird über formelle Vertragsbedingungen gestellt.[32][22]
4. **Reagieren auf Veränderung über Befolgen eines Plans:** Agile Praktiken betonen die Wichtigkeit von Flexibilität und unterstützen die Anpassung von Entwicklungsplänen an neue Erkenntnisse und veränderte Umstände.[32][22]

Agile Verfahren betonen den Stellenwert zyklischer Iterationen, die es Teams ermöglichen, in festgelegten Zeitabständen funktionstüchtige Softwareversionen bereitzustellen. Diese regelmäßigen Zyklen fördern eine fortwährende Überprüfung sowohl des Fortschritts in der Entwicklung als auch der Qualität des Produkts. Dadurch wird Teams die Möglichkeit gegeben, sich effizient in wandelbaren Umfeldern zu bewegen und rasch auf Veränderungen von Anforderungen zu reagieren.[22]

Die zwölf Prinzipien, die das Agile Manifest verkörpert, appellieren an kulturelle Umwandlung innerhalb der Entwicklerteams und der gesamten Organisation. Diese Prinzipien befürworten eine starke Ausrichtung auf Selbstverwaltung, Erreichen technischer Meisterschaft und Verfolgung von Simplität. Durch regelmäßige Selbstreflexion ihrer Arbeitsmethoden sind agile Teams in der Lage, ihre Arbeitsweise stetig zu optimieren und ein nachhaltiges Entwicklungstempo zu gewährleisten.[32]

2.7.2. Kanban

Kanban, das seine Wurzeln in der Produktionssteuerung des Lean Manufacturing hat, hat sich zu einem effektiven agilen Framework in der Welt der Softwareentwicklung gewandelt. In der Praxis unterstützt Kanban Softwareprojekte durch einen kontinuierlichen Prozess der Optimierung.[22]

- **Visualisierung des Arbeitsflusses:** Kanban erfordert die Darstellung aller Aufgaben in Form von Karten auf einem Kanban-Board, was den Überblick über den aktuellen Stand der Arbeit erleichtert.[22]
- **Begrenzung der laufenden Arbeiten:** Durch das Festlegen von Grenzen für gleichzeitig in Bearbeitung befindliche Aufgaben wird Überlastung vermieden und die Fokussierung auf Fertigstellung verbessert.[22]
- **Messung und Steuerung des Flusses:** Analyse von Durchlaufzeiten und Anpassungen der Arbeitsprozesse zur Steigerung der Effizienz.[22]
- **Explizite Prozessregeln:** Klare Richtlinien und Regeln sorgen für ein einheitliches Verständnis unter den Teammitgliedern.[22]
- **Feedbackschleifen:** Regelmäßige Meetings zur Überprüfung des Fortschritts und zur Identifikation von Verbesserungsmöglichkeiten.[22]
- **Verbesserung der Prozesse:** Förderung von Leadership und kontinuierlichen Verbesserungsprozessen auf allen Ebenen der Organisation.[22]

Indem Kanban im Bereich der Softwareentwicklung angewendet wird, unterscheidet es sich von konventionelleren, sequenziellen Ansätzen durch seine Förderung von Iterationen und Anpassungsfähigkeit, insbesondere im Umgang mit wechselnden Anforderungen und Prioritäten.

2.8. Wasserfall Modell

Das Wasserfallmodell, eine sequenzielle Entwurfsmethodik, die insbesondere in der Softwareentwicklung verbreitet ist, visualisiert den Entwicklungsfortschritt als einen kontinuierlichen, stufenweisen Fluss, der einem Wasserfall ähnelt. Es gliedert sich in distinkte Phasen: von der **Anforderungsanalyse** über den **Entwurf** und die **Implementierung** bis hin zum **Testen**, der **Integration** und schließlich der **Wartung**. Jeder dieser Schritte ist durch spezifische Liefsergebnisse und einen dedizierten Überprüfungsmechanismus charakterisiert, wobei eine strikte Trennung zwischen den einzelnen Phasen besteht – keine beginnt, bevor die vorherige nicht vollständig abgeschlossen ist. Im Vergleich zu agilen Entwicklungsmodellen (2.7) zeichnet sich das Wasserfallmodell durch eine rigide Struktur aus, die weniger Flexibilität bei der Anpassung an Veränderungen während des Entwicklungsprozesses bietet.[22]

3. Problemanalyse

In diesem Kapitel liegt der Fokus auf Schwierigkeiten, mit denen Programmierer hinsichtlich ihrer Arbeitsroutinen sowohl aus technischer Perspektive als auch auf der Ebene des Managements konfrontiert werden. Speziell im Bereich des Projektmanagements konzentriert sich die Diskussion auf zwei zentrale Methoden: den Wasserfallansatz und das agile Kanban-System. Obwohl diese Techniken dazu gedacht sind, die Softwareentwicklung zu vereinfachen und zu verbessern, stoßen Teams und Entwickler häufig auf Probleme, die ihre Effektivität beeinträchtigen. Es ist offensichtlich, dass kein Ansatz ideal ist. Jeder hat seine Vor- und Nachteile, abhängig von verschiedenen Faktoren wie dem Umfang des Projekts, dem Kundenfeedback und den Vertragsbedingungen.

Das Ziel dieses Kapitels besteht darin, die spezifischen Herausforderungen zu identifizieren, die mit den aktuellen Arbeitsabläufen verbunden sind. Durch das Verständnis dieser Probleme können Strategien entwickelt werden, um die Arbeitsabläufe für Entwicklerteams zu optimieren. Dabei sollen sowohl Projektmanagementtechniken als auch die Größe des Teams berücksichtigt werden.

3.1. Vorgehen der Analyse

Diese Arbeit verwendet eine methodische Kombination aus qualitativen und quantitativen Ansätzen, um die Wirksamkeit und Effizienz von aktuellen Entwicklungsworkflows von Entwicklungsteams zu untersuchen. Diese Methode ermöglicht eine umfassende Bewertung der Schwierigkeiten und Erfolgsfaktoren. Im Folgenden wird die dreistufige Analysemethode detailliert beschrieben.

3.1.1. Datenrecherche

Die Analyse beginnt mit einer umfassenden Recherche, um aktuelle statistische Daten zum Software-Entwicklungsprozess zu sammeln. Das Ziel besteht darin, relevante Werte wie Durchlaufzeiten, Budgettreue, Fehlerquoten, Branchenberichten und Erfahrungsberichte von anderen Entwicklern zu analysieren.

Die ausgewählten Daten bieten eine objektive Grundlage für die Bewertung der Effektivität und Effizienz der gängigen Methoden im Bereich der Softwareentwicklung. Durch die Analyse dieser Informationen kann ein umfassendes Verständnis dafür entwickelt werden, wie Workflows in realen Projekten aussehen, insbesondere in Bezug auf die Qualitätskontrolle und Einhaltung von Zeit- und Budgetvorgaben.

3.1.2. Strukturiertes Interview

Aufgrund der geringen Anzahl an Mitarbeitern im Bereich der Webentwicklung ist es nicht möglich, umfassende Umfragen durchzuführen. Stattdessen werden strukturierte Interviews mit einer ausgewählten Gruppe von Mitarbeitern durchgeführt. Ziel dieser Interviews ist es, einen tieferen Einblick in die praktische Anwendung, die wahrgenommenen Vor- und Nachteile sowie Verbesserungsmöglichkeiten zu gewinnen. Diese Interviews folgen einem strukturierten Rahmen, der darauf abzielt, durch fokussierte, im Voraus festgelegte Fragen ein präzises und unmittelbares Verständnis für das Thema zu gewährleisten.

3.1.3. Fallstudienanalyse

Neben den Interviews und statistischen Analysen wird zudem eine Fallstudienanalyse durchgeführt. Diese Analyse bezieht sich auf bereits vorhandene Literatur und Projektberichte um spezifische Erfolge und Herausforderungen zu identifizieren. Diese Phase der Untersuchung stützt sich auf akademische Publikationen und Projektberichte. Ziel ist es, durch die systematische Analyse dieser Quellen, tiefergehende Einsichten in praktische Umsetzung und Effektivität von Entwicklungsprozessen zu erlangen.

Dieses mehrdimensionale Analyseverfahren kombiniert quantitative und qualitative Methoden und soll ein tiefes Verständnis der Anwendung und Wirksamkeit von Methoden in der Softwareentwicklung schaffen. Das Mixed Method Research¹ Modell dient dabei als Orientierung. Die Ergebnisse dieser Analyse sollen als Grundlage für konstruktive Vorschläge zur Optimierung des Entwicklungsworkflows von Web-Entwicklungsteams dienen.

¹Das Mixed Method Research Modell kombiniert qualitative und quantitative Methoden um Forschungsfragen umfassender zu beantworten[9]

3.2. Analyse

Der Softwareentwicklungssektor ist dynamisch und anspruchsvoll. Er ist geprägt von einem hohen Tempo und ständigen Innovationen. Die Effizienz der Softwareentwicklungsabläufe ist für die digitale Infrastruktur in allen Bereichen von Bedeutung. Diese Arbeitsabläufe stellen auch Herausforderungen dar. Das Ziel dieser Untersuchung besteht darin, eine eingehende Analyse der Probleme vorzunehmen, mit denen die Softwareentwicklungsprozesse derzeit konfrontiert sind. Sie zielt darauf ab, die Gründe für ineffiziente Arbeitsprozesse zu identifizieren, die den Fortschritt in Softwareentwicklungsteams behindern. Die Herausforderung, Softwareentwicklungsprojekte rechtzeitig und innerhalb des Budgets abzuschließen, ist gut dokumentiert und unterstreicht ein anhaltendes Problem im Bereich der Informationstechnologie. Um das Ausmaß zu verdeutlichen ist die Betrachtung statistischer Daten von führenden Forschungs- und Beratungsunternehmen relevant.

3.2.1. Ausgewertete Daten und Statistiken

Eine Untersuchung der Harvard Business Review hebt die notorischen Schwierigkeiten bei der Verwaltung von IT-Projekten hervor und zeigt, dass im Durchschnitt Budgetüberschreitungen von 27 % auftreten. Diese Statistik zeigt deutlich, wie komplex und unvorhersehbar Softwareentwicklungsprozesse sind. Solche Überschreitungen haben weitreichende Folgen und wirken sich nicht nur auf die finanzielle Situation von Unternehmen aus, sondern auch auf ihre betriebliche Effizienz und Innovationskraft. [5].

Ein Bericht von McKinsey & Company in Zusammenarbeit mit der Universität Oxford zeigt, dass Softwareprojekte besonders anfällig für Budgetüberschreitungen sind. Im Durchschnitt überschreiten IT-Projekte mit einem Budget von über 15 Millionen Dollar das Budget um 45 % und den Zeitplan um 7 %, während sie 56 % weniger Wert liefern als prognostiziert. Diese Tendenz ist bei Softwareprojekten mit einer durchschnittlichen Budgetüberschreitung von 66 %, wie in Abbildung 3.1 dargestellt, besonders ausgeprägt. Die Überschreitung der geplanten Zeiträume bei IT-Projekten liegt mit 7 % deutlich über dem Durchschnitt, während Software-Projekte sogar eine Überschreitungsrate von 33 % aufweisen.[30]. Diese Zahlen deuten darauf hin, dass die bestehenden Verfahren in der Softwareentwicklung durch grundlegende Unzulänglichkeiten und Missverhältnisse geprägt sind, was zu erheblichen Abweichungen zwischen den antizipierten und den realisierten Ergebnissen der Projekte führt.

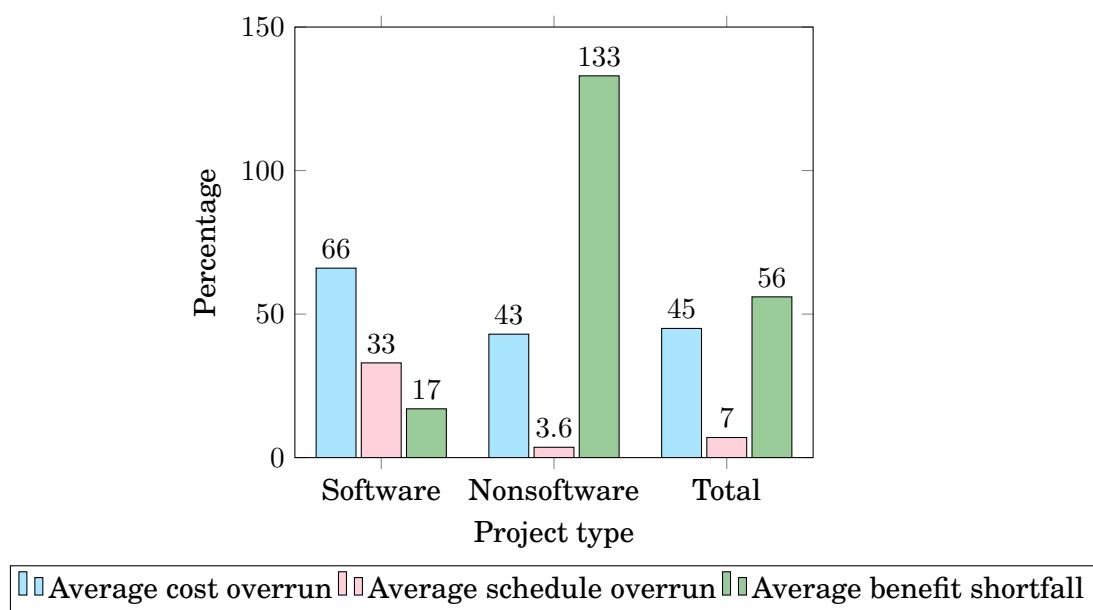


Abbildung 3.1.: Performance von unterschiedlichen IT-Projekten[30]

Der McKinsey-Bericht schlägt außerdem vier Schlüsselstrategien vor, um die Ergebnisse von IT-Projekten zu verbessern:

Konzentration auf das Management von Strategie und Stakeholdern², Beherrschung von Technologie und Projekthinhalten, Aufbau effektiver Teams und herausragende Projektmanagementpraktiken.

Diese Strategien unterstreichen den vielschichtigen Ansatz, der erforderlich ist, um die Herausforderungen von Budget- und Zeitüberschreitungen zu bewältigen, und

²alle Personen und Gruppen, die von den Entscheidungen und Aktionen eines Unternehmens betroffen sind oder beeinflusst werden können [33]

verdeutlichen die Bedeutung eines umfassenden Projektmanagements, des Engagements der Stakeholder, des technologischen Know-hows und der Teamdynamik bei der Bewältigung der Komplexität der Softwareentwicklung [30].

Im Bereich der IT-Projektmanagementmethoden zeigt der CHAOS³-Report 2015 der Standish Group aufschlussreiche Trends und Erfolgsquoten, die eine kritische Betrachtung der vorherrschenden Praktiken erfordern [19]. Die nachfolgende Grafik (Abbildung 3.2) visualisiert die Erfolgsraten von Projekten unterschiedlicher Größe, die entweder agile Methoden oder traditionelle Wasserfall-Modelle anwenden.

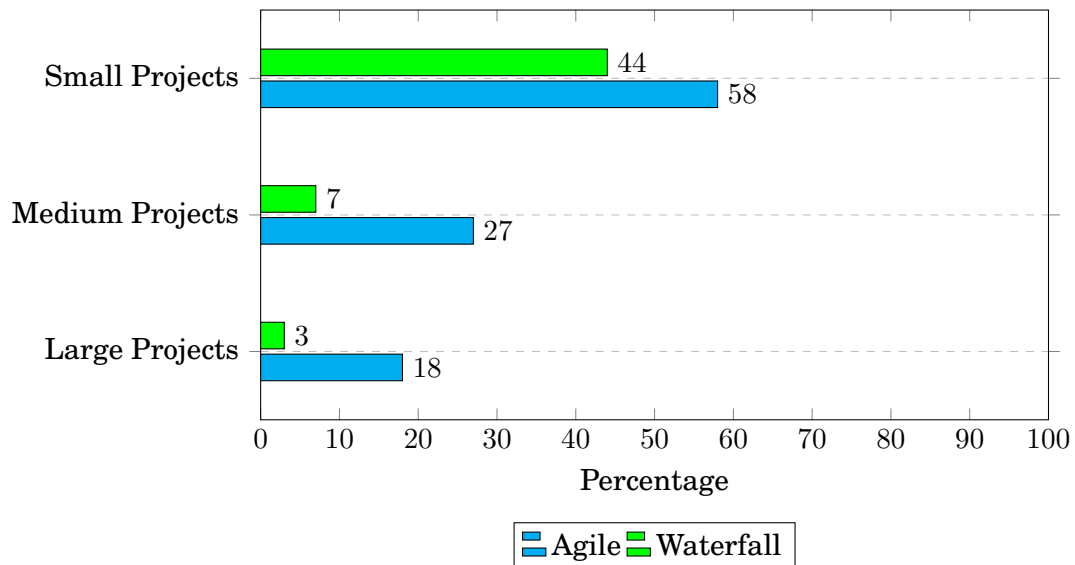


Abbildung 3.2.: Erfolgsrate von Projekten in Prozent[19]

Abbildung 3.2 offenbart eine deutliche Diskrepanz zwischen den Erfolgsraten von agilen Methoden und Wasserfall-Modellen über verschiedene Projektgrößen hinweg. Bei Großprojekten erreichen agile Methoden eine Erfolgsrate von 18 %, wohingegen Wasserfall-Modelle lediglich eine Quote von 3 % verzeichnen. Diese Tendenz setzt sich in Projekten mittlerer Größe fort, in denen agile Ansätze mit einer Erfolgsrate von 27 % im Vergleich zu 7 % bei Wasserfall-Modellen überzeugen. Bemerkenswerterweise zeigen auch bei kleinen Projekten, die für ihre Flexibilität und Anpassungsfähigkeit bekannt sind, agile Methoden mit 58 % eine höhere Erfolgsquote als Wasserfall-Modelle mit 44 %.

Derartige Statistiken legen die umfassenden Schwierigkeiten offen, mit denen IT-Projekte konfrontiert ist, und betonen die dringende Notwendigkeit, bestehende Projektmanagementmethoden kritisch zu hinterfragen und effektive Optimierungsstrategien zu entwickeln.

³Chaos Report ist eine Studie, die die Erfolgs- und Misserfolgsraten von IT-Projekten untersucht

Die Umfrage von Harvey Nash/KPMG Creative CIO [3] unter 3.352 Chief Information Officer (CIO) und Technologieverantwortlichen aus 82 Ländern zeigt hohe Misserfolgsraten bei der Implementierung von Customer Relationship Management (CRM)-Systemen (59 %), Enterprise Resource Planning (ERP)-Einführungen (62 %) und Big-Data⁴-Projekten (63 %). Diese Statistiken stützen sich auf Rückmeldungen von CIOs und zeigen signifikante Herausforderungen innerhalb von IT-Projekten. Die Statistiken von Teamstage⁵ zeigen, dass 70 % aller Projekte scheitern [36]. Joe Harley vom Department for Work and Pensions bestätigt diese Einschätzung und behauptet, dass nur 30 % der Projekte die gewünschten Ergebnisse erreichen.[6]

Die Ursachen für diese Misserfolgsraten wurden von IDC⁶ und der Application Developers Alliance [21] auf verschiedene Weise identifiziert. Ein großer Teil der Vorhaben scheitert aufgrund veränderter oder unzureichender Dokumentation der Anforderungen, unzureichender Finanzierung oder Ressourcenausstattung sowie fehlender Testzeit. Das Fehlen eines guten Organisationsmanagements und der Verlust wichtiger Mitarbeiter verstärken diese Faktoren. Sie weisen eher auf systemische Ineffizienzen im Arbeitsprozess hin als auf isolierte Vorfälle.

Obwohl die Untersuchung von Thamhain & Wilemon [44] im Jahr 1986 stattfand, bleibt sie auch heute noch relevant für die Schwierigkeiten des Projektmanagements. Besonders das Einhalten von Fristen, eine Herausforderung für 85 % der Projektmanager, sowie der Umgang mit Ressourcenbeschränkungen (83 %) stehen an der Spitze der Problematiken. Des Weiteren ist die effektive Kommunikation zwischen verschiedenen Arbeitsgruppen für 80 % der Befragten entscheidend. Die Beteiligung sowohl der Teammitglieder als auch des Managements und die Anpassungsfähigkeit an Veränderungen sind unvermeidbar, um Projekte erfolgreich zu leiten und zu realisieren.

Trotz des großen Lobes für ihre Flexibilität und Reaktionsfähigkeit steht die agile Methodik auch vor Herausforderungen, die ihren Erfolg beeinträchtigen können. Die neunte jährliche State of Agile™-Umfrage betont ein wichtiges Problem: Unzureichende Erfahrung mit agilen Praktiken ist für 44 % der Fehlschläge bei agilen Projekten verantwortlich [46]. Dieses Resultat zeigt, dass es eine bedeutende Hürde für den Einstieg gibt, vor allem für Teams, die erstmals in den agilen Ansatz einsteigen. Für eine erfolgreiche Umstellung auf Agile sind bestimmte Fähigkeiten und Kenntnisse erforderlich. Wenn sie nicht vorhanden sind, kann dies zu einer steilen Lernkurve mit möglichen Fehlschlägen führen [47].

Die Unternehmenskultur stellt ein weiteres bedeutendes Hindernis für die Implementierung von Agile dar: 42 % der Umfrageteilnehmer gaben an, dass es Konflikte zwischen den herkömmlichen Unternehmenswerten und dem fortschrittlichen Ethos

⁴"Unter Big Data versteht man Daten, die in großer Vielfalt, in großen Mengen und mit hoher Geschwindigkeit anfallen"[34]

⁵TeamStage ist eine Projektmanagement-App für Koordination und Produktivität

⁶International Data Corporation Alliance

von Agile gebe. Solch eine kulturelle Kluft zeigt sich oft in einer Abneigung gegen Neuerungen, was die Umsetzung und Effizienz agiler Verfahren hindern kann. [47]. Obwohl die Unterstützung agiler Prinzipien durch das Management als äußerst wichtig erachtet wird, identifizieren 38 % der Umfrageteilnehmenden den fehlenden Rückhalt seitens der Führungsebene als signifikantes Hindernis. Dieses Problem wird zusätzlich durch die Tendenz von 37 % verstärkt, sich an überholten Ansätzen, wie dem Wasserfallmodell, festzuklammern. Diese Kombination aus Herausforderungen schränkt die natürliche Flexibilität der Teams ein und untergräbt dadurch die Weiterentwicklung agiler Methoden.[47]

Das agile Paradigma legt großen Wert auf stabile Kommunikation und Zusammenarbeit. 33 % der befragten Personen äußerten, dass organisatorische oder kommunikative Barrieren eine reibungslose Implementierung agiler Verfahren beeinträchtigen. Diese Probleme können bestehende Herausforderungen intensivieren und die Wirksamkeit agiler iterativer Prozesse verringern [47]. Regelmäßiger Austausch über Pläne ist von entscheidender Bedeutung, insbesondere da agile Pläne häufigen Aktualisierungen unterliegen. Dies betont die Wichtigkeit, Teammitglieder und Stakeholder stets informiert zu halten und zu gewährleisten, dass Änderungen effektiv kommuniziert und implementiert werden. Solche Maßnahmen sind essenziell, um die Flexibilität und Reaktionsgeschwindigkeit des Teams zu steigern[7].

Die Wasserfall-Methode hingegen hat aufgrund ihrer sequentiellen Struktur mehrere Schwachstellen. Die Misserfolgsquote von Wasserfall-Projekten beträgt laut Boldare.com 29 %, während sie bei Agile-Projekten nur 9 % beträgt [23]. Da das schrittweise Vorgehen nur wenig Spielraum für Anpassungen und Verfeinerungen in der Mitte des Prozesses bietet, wird die Starrheit des Wasserfall-Prozesses, als wesentlicher Faktor für diese Diskrepanz betrachtet.

Die niedrigere Erfolgsquote von 49 % bei Wasserfall im Vergleich zu 64 % bei Agile ist ein weiterer Grund für diese Diskrepanz. Dies deutet darauf hin, dass die Projektziele nicht erreicht werden können. Diese Statistik zeigt, dass der strukturierte Ansatz von Wasserfall in einer Umgebung, die immer mehr Anpassungsfähigkeit und Fluidität erfordert, immer schwieriger wird [46].

Nach Angaben der Scrum Alliance [38] stellt das Fehlen klar definierter Erfolgsmetriken eine fortwährende Herausforderung bei der Umsetzung der agilen Transformation dar – eine Auffassung, die 27 % der Teams teilen. Laut dem 15th ANNUAL State of Agile™ Survey [46] spielen Metriken wie die Zufriedenheit der Kunden (59 %), der Geschäftswert (58 %), die erreichten Geschäftsziele (50 %), die rechtzeitige Lieferung (48 %), die Qualität (48 %), die Produktivität (41 %) und die Unternehmensmoral (41 %) eine entscheidende Rolle bei der Beurteilung der Agile-Effektivität. Diese Metriken werden identifiziert und überwacht, um die Komplexität von Agile-Projekten zu navigieren und ihre Ausrichtung an strategischen Zielen zu belegen.

Technical debt⁷ sind in der Softwareentwicklung bedeutende Herausforderungen. Der Begriff ist metaphorisch und beschreibt die kumulativen Folgen von Abkürzungen und ungünstigen Entwicklungsmethoden. Darunter fallen nicht nur bewusste Entscheidungen, die auf kurzfristige Gewinne auf Kosten der langfristigen Qualität abzielen, sondern auch unbeabsichtigte Anhäufungen, die aus mangelnder Voraussicht oder Verständnis resultieren. Eine Feldstudie zu technical debt von N. Ernst [15] zeigt die komplexe Natur (wie in Abbildung 3.3 zu sehen) und verschiedene Erscheinungsformen von technical debt auf, darunter fehlerhafte Entscheidungen in der Architektur und zu komplexe Code-Elemente, die sich im Laufe der Zeit zusammensetzen und die Systemwartung und -flexibilität beeinträchtigen.

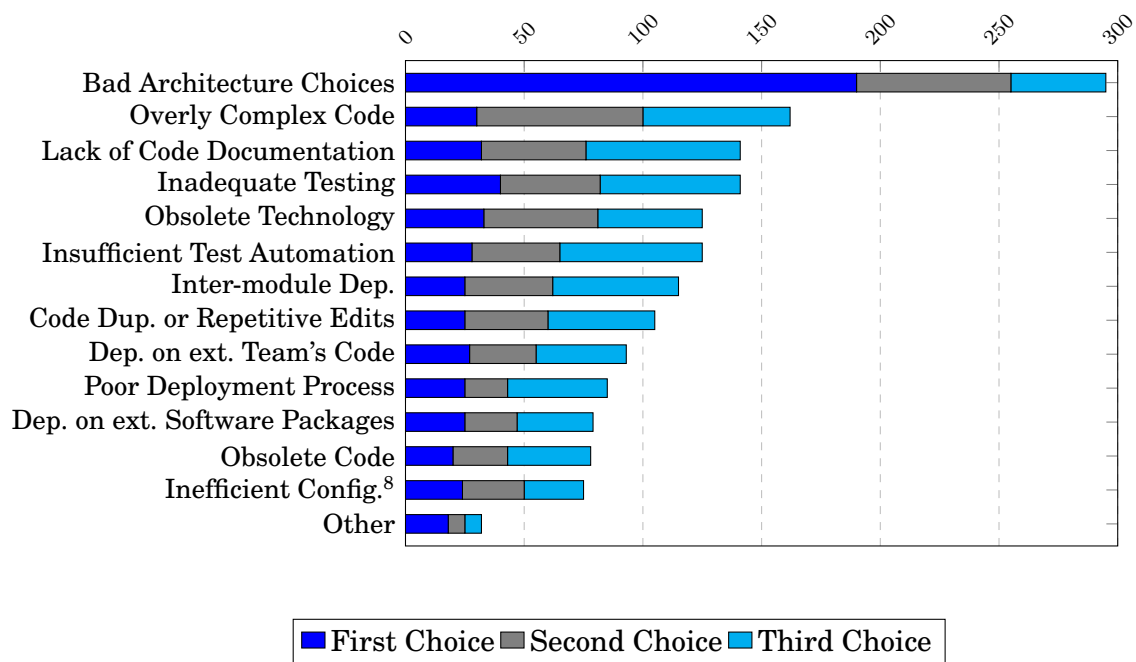


Abbildung 3.3.: Ranking Sources of Technical Debt [15]

Das Systems Sciences Institute von IBM beschreibt die Auswirkungen von technical debt auf die Kosten eindeutig [11]. Die Kosten für die Fehlerbehebung nach der Veröffentlichung können bis zu 100 Mal höher sein als bei der Erkennung und Behebung solcher Fehler während der Entwurfsphase. Die tiefere Verankerung von Fehlern im System und die umfassenderen Auswirkungen, wenn die Software zur Grundlage für unterschiedliche Abläufe wird, sind Gründe für diesen exponentiellen Kostenanstieg.

⁷Technical Debt is a metaphor [...] that frames how to think about dealing with this craft, thinking of it like a financial debt. The extra effort that it takes to add new features is the interest paid on the debt. (Martin Fowler)[17]

⁸Inefficient Configuration Management/Build Infrastructure

John Ousterhout argumentiert in seinem Buch "A Philosophy of Software Design" [35], dass die Unstimmigkeit der Software die Ursache für ihre Komplexität sei. Diese Diskrepanz wird besonders deutlich, wenn im Laufe der Zeit eine große Anzahl von Mitarbeitern, darunter auch Neulinge, die mit den etablierten Konventionen nicht vertraut sind, an einem Projekt arbeiten. Ousterhout betont die Bedeutung von Werkzeugen zur Erhaltung der Konsistenz und schlägt vor, automatische Prüfprogramme einzusetzen, um syntaktische Übereinstimmungen zu etablieren, bevor der Code an ein Repository übergeben wird.

Die Philosophie von Clean Code⁹: A Handbook of Agile Software Craftsmanship (Ein Handbuch für agiles Softwarehandwerk), die von Robert C. Martin vertreten wird, stimmt mit diesem Gedanken überein und befürwortet eine genaue Benennung von Variablen, ähnlich wie bei dem Namen eines Babys. Wie James O. Coplien im Vorwort schreibt: "You should name a variable using the same care with which you name a first-born child"[29]. Die Bedeutung von Klarheit und Präzision im Code wird durch diese Metapher hervorgehoben, da sie sich unmittelbar auf seine Verständlichkeit und Wartbarkeit auswirken.

⁹"Code is clean if it can be understood easily – by everyone on the team"[29]

Eine Umfrage über Clean-Code-Praktiken gibt Auskunft über das Verhalten der Entwickler in Bezug auf die Codequalität. Sie stimmt zu, dass es von Anfang an Schwierigkeiten gibt, sauberen Code zu schreiben, insbesondere unter dem Druck bevorstehender Fristen. Allerdings sind sich die Teilnehmer über die langfristigen Vorteile sauberen Codes bei späteren Änderungen bewusst (siehe 3.4). Der Zeitdruck am Ende eines Projekts kann die Qualität des Codes beeinträchtigen, wie die Umfrage zeigt. Diese wahrgenommene Begrenzung mindert jedoch statistisch gesehen nicht die Bedeutung der Einhaltung sauberer Codestandards.[25]

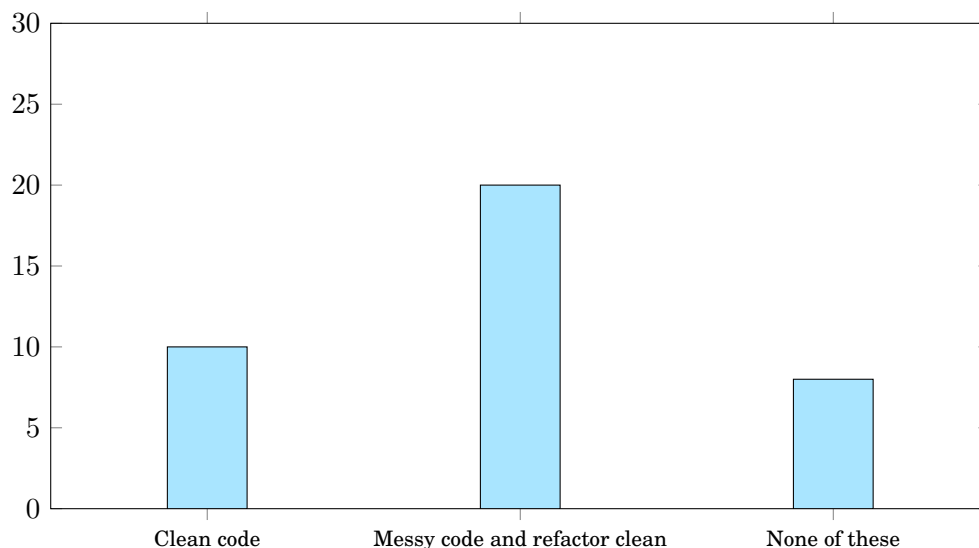


Abbildung 3.4.: Participant's perceptions regarding how they deal with Clean Code. [25]

Eine Umfrage über Clean-Code-Praktiken gibt Auskunft über das Verhalten der Entwickler in Bezug auf die Codequalität. Sie stimmt zu, dass es von Anfang an Schwierigkeiten gibt, sauberen Code zu schreiben, insbesondere unter dem Druck bevorstehender Fristen. Allerdings sind sich die Teilnehmer über die langfristigen Vorteile sauberen Codes bei späteren Änderungen bewusst (siehe 3.4). Der Zeitdruck am Ende eines Projekts kann die Qualität des Codes beeinträchtigen, wie die Umfrage zeigt. Diese wahrgenommene Begrenzung mindert jedoch statistisch gesehen nicht die Bedeutung der Einhaltung sauberer Codestandards.[25]

Die Effizienz von Arbeitsabläufen wird laut einer Untersuchung der Application Developers Alliance (2015) erheblich durch Faktoren wie schlechtes Management, ändernde Anforderungen oder Unterfinanzierung beeinträchtigt. Dies deutet darauf hin, dass diese Probleme nicht hauptsächlich unter direkter Kontrolle der Entwickler liegen (IDC, Application Developers Alliance, 2015) [21]. Ein weiteres Ergebnis der Studie zeigt, dass das Schreiben von qualitativ hochwertigem, performantem

und sicheren Code mit einer Zustimmungsrate von 43 % [21] eines der größten Probleme von Softwareentwicklern ist.

Außerdem weist die Zeitspanne, die für Tätigkeiten wie das Debuggen von unzureichendem Code verbraucht wird und ungefähr 42% der Zeit von Entwicklern beansprucht wird, auf technical debt in der Branche hin [46]. Die Kosten, die mit der Beseitigung von schlechtem Code verbunden sind, werden von Martin hervorgehoben: "Of course bad code can be cleaned up. But it's very expensive." (Natürlich kann schlechter Code bereinigt werden. Aber das ist sehr teuer.) [29]. Das verdeutlicht den Konflikt zwischen dem Aufwand für die Codequalität im Vorfeld und dem Wartungsaufwand auf lange Sicht.

Darüber hinaus weist der Bericht der Application Developers Alliance darauf hin, dass die verfrühte Freigabe von Software und der Zeitdruck für Entwickler den effizienten Arbeitsablauf erheblich beeinträchtigen. Diese Schwierigkeiten stimmen mit Martins Forderung überein, von Anfang an eine gründliche Codequalität sicherzustellen [29].

Es wird geschätzt, dass die Ausgaben für die Verwaltung von suboptimalem Code, zu denen sowohl direkte als auch indirekte Kosten gehören, wie etwa die rückläufige Arbeitsmoral, jährlich bis zu 85 Mrd. US-Dollar betragen. Das zeigt die Wichtigkeit von "Clean Code" bzw. hochwertigem Code.

Schlussfolgerung

Eine gründliche Auswertung der gesammelten Daten und Statistiken hat sowohl tiefe Einblicke in die Probleme als auch die Wirksamkeit des derzeitigen Ablaufs bei der Entwicklung von Webprojekten geliefert. Diese Analyse offenbart, dass das Überschreiten von Budgets und Zeitplänen in IT-Vorhaben ein allgegenwärtiges Phänomen ist, was die dringende Notwendigkeit für einen verfeinerten Ansatz zeigt. Die Erkenntnisse, speziell jene, die aus Studien der Harvard Business Review sowie dem McKinsey & Company Bericht hervorgehen, zeigen die Herausforderungen, die mit der Einhaltung finanzieller und zeitlicher Vorgaben verbunden sind. Dabei zeigen die durchschnittlichen Überschreitungen von Budgets ein gravierendes Hindernis für die Effektivität und ökonomische Tragfähigkeit von Entwicklungsprojekten auf. Der Vergleich der Erfolgsquoten zwischen Projekten, die agile Methoden nutzen, und solchen, die auf traditionelle Wasserfall-Modelle setzen, hebt die Überlegenheit agiler Ansätze in Bezug auf Flexibilität und Adaptionfähigkeit hervor. Allerdings offenbart diese Gegenüberstellung auch, dass selbst innerhalb der agilen Praxis Verbesserungspotenziale im Entwicklungsablauf existieren. Dies betont zwar die Bedeutung des Einsatzes agiler Techniken zur Erhöhung der Anpassungsfähigkeit an veränderliche Anforderungen und zur Effizienzsteigerung im Projektmanagement, macht aber ebenso deutlich, dass die Agile Methodik alleine nicht als ultimative Lösung für alle Schwierigkeiten im Entwicklungsprozess gesehen werden kann.

Die Untersuchung des Konzepts technical debt unterstreicht die langfristigen Konsequenzen, die durch Abstriche bei der Codequalität entstehen können, welche wiederum die Wartbarkeit und Skalierbarkeit der Software negativ beeinflussen. Dies betont die Wichtigkeit, bewährte Verfahrensweisen in der Codeerstellung nicht nur zu implementieren, sondern auch konsequent anzuwenden, um die Entstehung von technical debt so weit wie möglich zu reduzieren. Es ist dabei kritisch zu betonen, dass Einbußen in der Codequalität, selbst unter dem Druck von zeitlichen Begrenzungen, unbedingt vermieden werden sollten, da sie langfristige negative Auswirkungen auf das Projekt haben und dessen Nachhaltigkeit gefährden können.

Vor dem Hintergrund dieser Einsichten wird klar, dass die Feinabstimmung des Entwicklungsprozesses nicht nur für die Steigerung der Effizienz und der Qualitätsstandards eine entscheidende Rolle in der Softwareentwicklung spielt, sondern auch maßgeblich dazu beitragen kann, Überschreitungen des Budgets und des Zeitrahmens zu minimieren.

3.2.2. Interview mit Tech Lead Paul Schnabel

Um die Leistungsfähigkeit von Arbeitsabläufen in der Webentwicklung eingehend zu analysieren und umfassende Erkenntnisse über die Nutzung im Alltag, die wahrgenommenen Stärken und Schwächen sowie potenzielle Optimierungsansätze der gegenwärtigen Arbeitsmethoden zu erlangen, wurde ein ausführliches Gespräch mit Tech Lead Paul Schnabel geführt.

Methodik

Das Gespräch mit Paul Schnabel wurde anhand eines strukturierten Leitfadens geführt, der aus vorher festgelegten Fragen bestand. Dieser Ansatz zielte darauf ab, eine direkte und präzise Einsicht in das behandelte Thema zu erlangen. Im Fokus des Interviews standen verschiedene Kernpunkte, darunter die allgemeine Zufriedenheit mit dem bestehenden System, die Auswahlprozesse von Arbeitsabläufen und Methodiken, die Vorteile des momentanen Workflows, Potenziale für Verbesserungen, die Aufgeschlossenheit gegenüber neuen Methoden, Erfahrungen mit alternativen Vorgehensweisen sowie der Bedarf an weiterer Unterstützung.

Ergebnisse und Analyse

Allgemeine Zufriedenheit und Bewertung

Paul Schnabel bewertet den aktuellen Workflow mit einer 3 von 10, hauptsächlich wegen des Fehlens eines klar definierten Workflows, was effizientes Arbeiten und Erfolgsmessungen erschwert.

Wahl von Workflow und Methoden

Jira Kanban Board wird genutzt, um die Arbeitsabläufe anzupassen und den Fortschritt zu verfolgen, was zur Teamdynamik und Aufgabenflexibilität beiträgt. Dieses Tool unterstützt eine effiziente Gestaltung des Workflows.

Stärken des aktuellen Workflows

Die hohe Flexibilität des aktuellen Workflows ermöglicht es, effektiv auf Änderungen zu reagieren und unkonventionelle Richtungen einzuschlagen, was die Produktivität und Effizienz steigert.

Verbesserungspotential

Trotz der grundsätzlichen Funktionalität des Workflows besteht Verbesserungspotential in der Planbarkeit und Effizienz der Prozesse.

Offenheit für neue Ansätze

Es besteht eine hohe Offenheit für die Einführung neuer Methoden, um Effizienz und Geschwindigkeit der Abläufe zu verbessern, wobei Bedenken hinsichtlich der Akzeptanz neuer Methoden bei den Angestellten bestehen.

Erfahrungen mit alternativen Ansätzen

Es liegen Erfahrungen mit spezialisierten Teams und Fullstack-Konzepten vor, wobei beide Ansätze Vor- und Nachteile in Bezug auf Koordination und Kompetenzspektrum zeigen. Auch wurde im Rahmen eines Kundenprojekts bereits ein Codegenerierungstool verwendet, was sich als effektiv erwiesen hat.

Unterstützungsbedarf

Für die erfolgreiche Implementierung neuer Methoden oder Anpassungen im Workflow werden vor allem unterstützende Werkzeuge benötigt.

Schlussfolgerungen und Implikationen

Das Gespräch mit Paul Schnabel zeigt, dass, obwohl der derzeitige Arbeitsprozess Vorteile wie Flexibilität und Adaptivität aufweist, Potential für Verbesserungen ungenutzt bleibt. Die Bereitschaft, sich neuen Verfahren zu öffnen, bietet neue Möglichkeiten, die Arbeitsabläufe in der Webentwicklung zu verfeinern. Diese Erkenntnisse tragen wesentlich dazu bei, die spezifische Möglichkeiten zur Verbesserung zu identifizieren und die Notwendigkeit der Integration innovativer Methoden und Instrumente hervorzuheben.

(Das volle Interview findet sich unter Anhang A)

3.2.3. Auswertung Fallstudien

Nach der Durchsicht umfassender Datensammlungen und den daraus gewonnenen Einsichten durch strukturierte Interviews, richtet sich der Fokus dieses Abschnitts auf die Analyse von Fallstudien. Diese Untersuchung stellt einen wesentlichen Bestandteil dieser Arbeit dar, da sie vertiefte Perspektiven auf die Anwendbarkeit und Wirksamkeit von Prozessen in der Softwareentwicklung eröffnet. Ziel ist es, durch die Analyse ausgewählter Fallstudien – die sich aus akademischen Forschungsarbeiten sowie ausführlichen Berichten über Projekte zusammensetzen – spezielle Schwierigkeiten, die in der Softwareentwicklung auftreten, zu erkennen und deren Ursachen zu identifizieren.

A Systematic Review of Productivity Factors in Software Development

Stefan Wagner und Melanie Ruhe haben in ihrer umfangreichen Studie die vielschichtigen Einflüsse, die die Produktivität in der Softwareentwicklung prägen, aus einem breiten Blickwinkel untersucht. Dabei richteten sie ihr Augenmerk nicht allein auf technische Aspekte, sondern in gleichem Maße die Rolle menschlicher Elemente wahrnimmt. Diese Herangehensweise ermöglichte es ihnen, ein umfassendes Verständnis der Faktoren zu entwickeln, die Effizienz und Effektivität in Entwicklungsprojekten antreiben. Ihre Arbeit, die sowohl offensichtliche als auch feinere Einflussgrößen beleuchtet, stellt eine grundlegende Ressource dar. Sie bietet tiefe Einblicke in die komplexen und wechselseitigen Faktoren, die die Leistungsfähigkeit im Bereich der Softwareentwicklung bestimmen[48].

Wagner und Ruhe haben ihren Ansatz auf einer systematischen Übersicht der Fachliteratur aufgebaut und setzen diesen in Beziehung zu Methoden, die auf empirischer Datenerhebung basieren. Ihre ausführliche Analyse der existierenden Literatur bietet einen umfassenden Einblick in die identifizierten Faktoren, die die Produktivität beeinflussen. Im Kontrast dazu fokussiert sich die empirische Forschung vornehmlich auf Daten, die aus konkreten Projekten oder Organisationen gewonnen wurden. Obwohl sie tiefgreifende Einsichten in einzelne Szenarien liefert, erreicht sie möglicherweise nicht die umfassende Perspektive, die durch eine Literaturübersicht geboten wird [48].

Das Paper leitet mit einer essentiellen Herausforderung im Feld der Softwareentwicklungsproduktivität ein: der Stützung auf mangelhafte empirische Daten. Es hinterfragt kritisch die gängige Praxis, sich auf "oberflächliche Umfragen und qualitative Erfahrungsberichte" zu verlassen, welche die Vielschichtigkeit der Einflussfaktoren auf die Produktivität in der Softwareentwicklung nicht vollständig abbilden. Diese Kritik wird mit einem Verweis auf die Arbeiten von Frese und Brodbeck (Quelle 12, Seite 1)[48] verstärkt, welche die Debatte über Arbeitsumstände in der Softwareentwicklung und Produktivitätsfaktoren als auf einer unzureichenden empirischen Basis stehend sehen.

Die Analyse zeigt eine Forschungslücke auf, welche die Tendenz offenlegt, den Schwerpunkt vornehmlich auf technische Faktoren (z. B. Größe der Software und deren Komplexität) zu legen, während wichtige menschliche Aspekte wie Meetings, die Organisation von Projekten und die Qualifikation der Entwickler (Quelle 7, Seite 1)[48] vernachlässigt werden. Diese Erkenntnis ist von Bedeutung, da sie die Möglichkeit aufzeigt, dass ein erheblicher Teil der Einflüsse auf die Produktivität der Entwickler übersehen wird, sollte die Forschung sich ausschließlich auf technische Elemente beschränken. Die Forderung nach einer Betrachtung, die sowohl technische als auch nicht-technische Faktoren einbezieht, ist wichtig, um ein vollständiges Verständnis für den Entwicklungsworkflow in der Softwareentwicklung zu erlangen. Softfaktoren wie Teamdynamiken und individuelle Kompetenzen werden im Vergleich zu technologischen Faktoren weniger intensiv erforscht, obwohl sie auch zur Produktivität beitragen. Der vorliegende Artikel setzt sich für eine stärkere Einbeziehung von Kommunikationsprozessen und anderen menschlichen Faktoren ein, die für den Erfolg von Softwareprojekten entscheidend sind. Diese Forderung findet Rückhalt in den empirischen Befunden diverser Untersuchungen, unter anderem jener von Brodbeck (Quelle 6, Seite 3)[48], welche die positive Beziehung zwischen Kommunikationsintensität und Projekterfolg herausstellt. Damit wird die gängige Vorstellung, dass intensiver Austausch die Produktivität hemmt herausgefordert. Eine Aufschlüsselung der Softfaktoren findet sich in Tabelle 3.1

Tabelle 3.1.: Softfaktoren und ihre Auswirkungen auf die Produktivität[48]

Faktor	Beschreibung	Quellen
Kommunikation	Wie effizient Informationen im Team ausgetauscht werden	4
Teamkohäsion	Die Kooperativität der Stakeholder und wie gut das Team zusammenarbeitet	9
Turnover	Die Menge an Personaländerungen im Team	7
Programmierer-Fähigkeit	Die Skills des Entwicklers	10
Zeitplan	Die Angemessenheit des Zeitplans zur Entwicklung	5
Anforderungsstabilität	Die Anzahl der Anforderungsänderungen	6
Teamgröße (Durchschnitt)	Anzahl der Personen im Team	10

Eine überraschende Einsicht, die aus den betrachteten empirischen Untersuchungen gewonnen wurde, ist die vergleichsweise geringe Wichtigkeit der Erfahrung im Gegensatz zu den Kompetenzen der Entwickler. Dies stellt die weit verbreitete Vermutung in Frage, umfangreichere Erfahrung führe unmittelbar zu gesteigerter Produktivität (Quelle 4, Seite 3).

Die Beispiele zeigen, wie menschliche Interaktionen, Teamdynamik und organisatorische Rahmenbedingungen die Produktivität stark beeinflussen. Obwohl diese Faktoren nicht immer unmittelbar messbar sind, sind ihre Auswirkungen bedeutend und können über Erfolg oder Scheitern von Softwareprojekten entscheiden.

In ihrer detaillierten Untersuchung legen Wagner und Ruhe dar, dass neben den menschlichen und organisatorischen Aspekten, technische Faktoren wie "Produktkomplexität" und "Zeitbeschränkungen in der Entwicklung" ebenfalls kritische Faktoren für die Produktivität darstellen. Solche Faktoren wurden in Forschungsarbeiten wie COCOMO II von Boehm und Kollegen (Seiten 3-4, Quelle 4)[48] analysiert und haben sich als essentiell für das Verstehen und die Optimierung von Softwareentwicklungsprozessen herausgestellt. Ebenso schreibt Jones (Seiten 3-4, Quelle 18)[48] die Relevanz von etwa 250 Faktoren. Brooks greift in seiner Studie bei IBM auf die von Walston und Felix identifizierten Faktoren zurück (Seite 2, Quelle 30)[48] und hebt insbesondere die Bedeutung der Programmkomplexität und strukturierten Programmierung hervor. Eine Aufschlüsselung der technischen Faktoren findet sich in Tabelle 3.2.

Tabelle 3.2.: Technische Faktoren und ihre Auswirkungen auf die Produktivität[48]

Faktor	Beschreibung	Quellen
Produktkomplexität	Wie komplex ist die Funktion und Struktur der Software?	6
Beschränkung der Entwicklungszeit	Wie viel der verfügbaren Ausführungszeit wird verbraucht?	7
Wiederverwendbarkeit	Inwieweit sollten die Komponenten wiederverwendbar sein?	3
Dokumentationsanpassung an Lebenszyklusbedürfnisse	Wie gut entspricht die Dokumentation den Bedürfnissen?	2

Diese technischen Aspekte spielen eine zentrale Rolle beim Scheitern von Projekten, da sie grundlegende Beschränkungen und Herausforderungen innerhalb der Softwareentwicklung aufzeigen. Zum Beispiel kann eine ausgeprägte Produktkomplexität, falls sie nicht korrekt gehandhabt und strukturiert wird, die Entwicklungszeiten erheblich in die Länge ziehen, was wiederum Budgetüberschreitungen und Abwei-

chungen vom Projektplan zur Folge haben kann. Ebenso kann eine mangelnde Anpassung der Dokumentation an die Erfordernisse des Produktlebenszyklus zu einem Verlust wichtiger Informationen führen, wodurch es schwieriger wird, das gesamte Team auf dem Laufenden und koordiniert zu halten.

Die Analyse von Wagner und Ruhe zeigt auf, dass die Produktivität in der Softwareentwicklung durch eine Mischung aus technischen und nicht-technischen Faktoren beeinflusst wird. Gemäß den Ausführungen von DeMarco und Lister (Quelle 11) sowie Hatton (Quelle 14), ist die Kompetenz der Entwickler von zentraler Bedeutung. Zudem bestehen signifikante Unterschiede in der individuellen Performance, was auf Potenzial für Verbesserungen in Entwicklungsprojekten deutet. Die Notwendigkeit, diese Einflussgrößen zu identifizieren und zu quantifizieren, um die Produktivität wirksam zu fördern, wird von den Autoren hervorgehoben. Das systematische Review bietet eine Liste wesentlicher Produktivitätsfaktoren, die als Ausgangspunkt für die Modellierung von Produktivitätsstrategien und für Ansätze zur Steigerung der Produktivität dienen kann.

Optimization in Software Engineering - A Pragmatic Approach

Die Optimierung in der Softwaretechnik ist ein Kernanliegen, das darauf abzielt, die Entscheidungsfindung für die zahlreichen Faktoren, die zum Erfolg eines Projekts beitragen, zu verfeinern. Günther Ruhes Studie "Optimization in Software Engineering - A Pragmatic Approach" gewährt umfassende Einblicke in die Herausforderungen und pragmatischen Lösungsansätze in diesem Feld.

Ruhe identifiziert das Manövrieren durch Entscheidungsumgebungen, die von unvollständigen, unpräzisen und zuweilen konträren Informationen geprägt sind, als eine zentrale Herausforderung in der Softwareentwicklung. Er führt aus, dass die grundlegende Komplexität und der Mangel an Informationen signifikante Barrieren für den Entscheidungsprozess darstellen, die zu weniger optimalen Entscheidungen führen können, welche wiederum negative Auswirkungen auf die Projektergebnisse haben([37], Seite 2). Angesichts dieser Komplexität ist ein differenzierter Optimierungsansatz erforderlich, der ein tiefes Verständnis für die Beschränkungen der vorhandenen Daten voraussetzt.

Die Untersuchung hebt hervor, dass Softwareentwicklung ihrem Wesen nach ein kreativer und menschenorientierter Prozess ist, der Unvorhersehbarkeiten und Subjektivität in Entscheidungsfindungsprozesse einführt. Ruhe ist der Meinung, dass die menschliche Natur der Softwareentwicklung einen differenzierten Ansatz erfordert, der ihre Unvorhersehbarkeit bedenkt, weil Menschen sich nicht immer nach rationalen Argumenten richten ("Software development is a creative and human-centered process, and humans do not necessarily act based on rational arguments.") ([37], Seite 2). Dieses Erkenntnis betont die Wichtigkeit von Flexibilität und der Beachtung menschlicher Aspekte in den Optimierungsstrategien.

Ruhe macht auf die Komplexität aufmerksam, die aus der Notwendigkeit resultiert, Entscheidungen aufgrund einer Vielzahl von Kriterien zu fällen. Dies verlangt eine Abwägung unterschiedlicher Aspekte und erschwert die Bestimmung der "optimalen" Entscheidung unter mehreren erstrebenswerten Resultaten ("Improving in one direction typically requires compromising against another criteria") ([37], S.2). Die Erfordernis, Kompromisse zu finden, fügt dem Optimierungsprozess eine zusätzliche Ebene der Komplikation hinzu und erfordert ein feinfühliges Austarieren zwischen den widerstreitenden Interessen des Projekts.

Ruhes Checkliste für den empfohlenen Prozess und dessen Anwendung ist ein zentrales Element dieses Rahmens, das die systematische Annäherung an Optimierungsaufgaben in der Softwareentwicklung erleichtert.

Im Folgenden werden die Schlüsselpunkte der Checkliste mit einer kurzen Beschreibung vorgestellt: ([37], Seite 5-9)

1. Scoping¹⁰ and Return on Investment (ROI)¹¹ Analysis:

Dieser Schritt definiert den Kontext des Problems und seine Grenzen. Die Durchführung einer ROI-Analyse zu diesem frühen Zeitpunkt hilft dabei, Tiefe und Breite der Untersuchung zu bestimmen. Dies ist entscheidend, um sicherzustellen, dass die Optimierungsbemühungen in Bereichen mit dem höchsten potenziellen Wert konzentriert werden.

2. Modeling and Problem Formulation:

In dieser Phase werden die Variablen, Einschränkungen und Ziele des Problems formuliert. Es ist wichtig, die Formulierung gegen die ursprüngliche Problemstellung zu verifizieren, um sicherzustellen, dass das Modell das Problem genau abbildet und damit eine solide Basis für die Optimierung bildet.

3. Solution Design:

Die Designphase beinhaltet eine Analyse der Fragen "Was?" und "Wie viel ist genug?". Die Erkundung möglicher Lösungsalternativen und der damit verbundenen Werkzeuge hängt vom gewählten Umfang ab. Dieser Schritt entscheidet über den Einsatz der Methoden, anhand der Größe und Art des Problems sowie seiner projizierten Auswirkung.

4. Data Collection:

Die Sammlung und Vorbereitung der für die Optimierung benötigten Daten ist entscheidend. Empirische Studien können genutzt werden, um die Menge und Qualität der Daten zu verbessern. Zielorientierte Messungen leiten die Datensammlung und -analyse.

¹⁰Vorhersage des Gesamtaufwands, der mit einem Projekt verbunden ist.

¹¹Kapitalrendite

5. Optimization:

Um das Werkzeug und den Algorithmus zur Optimierung anzupassen, müssen die Parameter genau festgelegt werden. Dazu gehört auch, die Robustheit der Lösung gegenüber Änderungen der Eingaben und die Auswirkungen neuer oder geänderter Einschränkungen zu bewerten

6. Validation:

Die Lösung des Optimierungsproblems muss validiert werden. Die Ergebnisse werden mit alternativen Algorithmen verglichen, um die Angemessenheit für das ursprüngliche Problem zu überprüfen.

7. Implementation:

Die ausgewählte Lösung wird als Entscheidung für das ursprüngliche Problem implementiert, was einen kritischen Schritt von der Theorie zur Praxis darstellt.

8. Evaluation:

Die Nützlichkeit der implementierten Lösung wird im Kontext des ursprünglichen Problems und mit den beteiligten Stakeholdern evaluiert, um die Gesamtwirksamkeit der Optimierungsanstrengungen zu messen.

Die Softwareentwicklung zeichnet sich durch ihre einzigartigen Herausforderungen aus, besonders wenn es um Entscheidungsprozesse und die Implementierung von Verbesserungen geht.

Im Kern dieser Disziplin stehen mehrere Schlüsselfaktoren: ([37], Seite 16)

1. Die Unsicherheit bezüglich des Umfangs von Softwareprojekten und -produkten ist beträchtlich.
2. Die Planung und Kalkulation von Softwarevorhaben gestalten sich schwierig, oft bedingt durch vage Anforderungen oder das Fehlen von Informationen.
3. Der nicht-deterministische Charakter der Softwareentwicklung, gekennzeichnet durch unvollständige und manchmal gegenläufige Daten, fügt eine weitere Ebene der Komplexität hinzu.
4. Die objektive Bewertung und Quantifizierung der Qualität von Software stellt eine beachtliche Herausforderung dar.

Abschließend ergibt sich die Notwendigkeit, zu prüfen, welche Art der Optimierung in welchem Kontext am geeignetsten ist und in welchem Maße dieser Weg verfolgt werden sollte. Ist mehr immer besser? ([37], Seite 16)

Optimierung stellt ein zentrales Instrument dar, um die Entwicklung und Weiterentwicklung von Software zu fördern. Das Ziel der Prozessoptimierung steht im Mittelpunkt der Bemühungen von Organisationen, die das höchste Reifestadium, Capability Maturity Model Integration (CMMI)-Level 5¹², erreicht haben. Die Anwendungsmöglichkeiten der Optimierung in der Softwaretechnologie sind nahezu unbegrenzt. Allerdings muss deren Anwendung gesteuert werden. Es ist nicht nur von Bedeutung, den Anwendungsbereich der Optimierung zu erweitern, sondern auch die Kosten zu betrachten. ([37], Seite 21) Optimierung darf jedoch nicht als universelles Lösungsmittel missverstanden werden. Der Fokus gilt nicht ausschließlich den Zahlen; vielmehr liegt das Interesse an tiefgreifenden Erkenntnissen, insbesondere an solchen, die in praktische Handlungen umgesetzt werden können. Eine wohlüberlegte umgesetzte Optimierung kann einen wertvollen Beitrag zur Entscheidungsfindung leisten. Ihre pragmatische Anwendung erfordert ein Verständnis für den Umfang und das Ausmaß ihrer Nutzung. ([37], Seite 22)

Communication Challenges in Agile Teams from The Communication Theory Perspective

Die agile Softwareentwicklung legt ihren Fokus auf Anpassungsfähigkeit, enge Zusammenarbeit mit dem Kunden und die Offenheit für Änderungen der Projektanforderungen, selbst wenn diese erst spät im Entwicklungsverlauf auftreten. Ein zentrales Leitbild des agilen Ansatzes ist die Priorisierung von Menschen und deren Interaktionen vor festgelegten Prozessen und Tools. Die Studie "Communication Challenges in Agile Teams from The Communication Theory Perspective" von Yermolaieva hebt hervor, wie kulturelle, sprachliche, organisatorische und technologische Barrieren die Teamkohäsion und Produktivität behindern können [50].

Im Rahmen der Studie wurde eine empirische Erhebung durchgeführt, an der 53 Personen teilnahmen. Ziel dieser Umfrage war es, die vorherrschenden Kommunikationsherausforderungen innerhalb agiler Teams zu identifizieren. Die Teilnehmer wurden aufgefordert, die drei ihrer Erfahrung nach häufigsten Schwierigkeiten zu benennen, mit denen sie in ihrem agilen Arbeitsumfeld konfrontiert wurden [50].

Aus der Umfrage ging hervor, dass eine Reihe von Kommunikationshürden existiert, wobei der "Projektmanagementprozess" und die "Persönliche Praxis" mit jeweils 16 Erwähnungen die Spitzenpositionen einnahmen, dicht gefolgt von "Koordination" mit 14 Erwähnungen. Diese Befunde verdeutlichen die Existenz zweier primärer Kategorien von Kommunikationsproblemen: strukturelle und personenbezogene. Unter Tabelle 3.3 findet sich eine Aufschlüsselung der für diese Arbeit relevanter Punkte, da sich nicht auf geographischen Schwierigkeiten konzentriert wird [50].

¹²Kennzeichnet Unternehmen, die der kontinuierlichen Verbesserung Vorrang einräumen

Die strukturellen Herausforderungen umfassen Aspekte der Teamorganisation und -führung. In diese Kategorie fallen der Projektmanagementprozess, Koordinationsprobleme und die Organisationskultur, was darauf hinweist, dass ineffiziente Abläufe und eine defizitäre Unternehmenskultur die Kommunikation innerhalb agiler Teams erheblich stören können.

Auf der anderen Seite stehen die personenbezogenen Herausforderungen, die sich auf die individuellen Verhaltensweisen und Einstellungen der Teammitglieder beziehen. Hierzu zählen die persönliche Arbeitsweise, Sprachbarrieren und das gegenseitige Vertrauen im Team, welche auf individueller Ebene adressiert werden müssen, um die Kommunikation effektiv zu gestalten.

Tabelle 3.3.: Kommunikationsprobleme in agilen Teams [50]

Challenge	Nennungen
Teamgrößen	13
Koordination	14
Projektmanagementprozess	16
Organizational factors	14

Die Kategorisierung der ermittelten Schwierigkeiten anhand bestimmter Kommunikationstheorien ermöglicht ein umfassendes Verständnis und legt den Grundstein für Lösungsansätze. Zum Beispiel veranschaulicht die "Symbolic Interaction Theory"¹³ die Wichtigkeit eines gemeinsamen Verständnisses und der individuellen Perspektiven, was die Bedeutung unterstreicht, sich gezielt mit individuellen Arbeitsweisen und Sprachbarrieren auseinanderzusetzen [50].

¹³Symbolic interactionism ist eine Theorie auf der Mikroebene, die sich auf die mit der menschlichen Interaktion verbundenen Bedeutungen konzentriert.[24]

3.3. Ergebnis der Analyse

Die eingehende Untersuchung der Prozesse in der Softwareentwicklung hat wesentliche technische sowie methodische Schwierigkeiten zu Tage gefördert. Ins Auge fallen dabei vor allem die Neigung zu Überschreitungen des Budgets und des Zeitrahmens, Probleme im Management von technical debt und die kritische Rolle, die menschliche Aspekte für die Projektperformance spielen. Wesentliche Erkenntnisse:

1. **Technical debt:** Die Analyse statistischer Daten zeigt regelmäßige Budget- und Zeitüberschreitungen auf, was ein Indiz für die inhärente Komplexität von Softwareprojekten ist.
2. **Methodische Schwachstellen:** Es zeigt sich, dass sowohl agile Methoden als auch das Wasserfall-Modell ihre eigenen Mängel aufweisen, was sich in unterschiedlichen Erfolgsquoten niederschlägt.
3. **Problematik technical debt:** Das Anhäufen von technical debt stellt ein ernstzunehmendes Hindernis dar, das die Beständigkeit und Qualität von Softwareprodukten nachteilig beeinflusst.
4. **Menschliche Faktoren:** Teamdynamik, effektive Kommunikation und Fachexpertise ist entscheidend für den Projekterfolg.

Die Ergebnisse heben hervor, wie entscheidend es ist, bei der Verfeinerung der Entwicklungsabläufe sowohl Technik als auch menschliche Dynamiken einzubeziehen. Diese Erkenntnis wird als Leitfaden dienen, um effektive Lösungsstrategien zu entwerfen, die in den folgenden Kapiteln umfassend erörtert werden.

Aufgrund der Ergebnisse der Analyse wird als nächster Schritt ein Lösungsansatz entwickelt, der die identifizierten Probleme berücksichtigt und die Prozesse in der Softwareentwicklung verbessert.

4. Lösungskonzept

Wie in Kapitel 3.2 Problemanalyse festgestellt, hat der Softwareentwicklungsprozess einige Schwächen, wie technical debt, Codequalität, methodische Schwachstellen und Softfaktoren wie Kommunikation, bzw. Menschen allgemein, aber auch Verzögerung und Kostenüberschreitung.

Eine effiziente Kommunikation über die Architektur und Funktionsweise von APIs ist in der Softwareentwicklung unverzichtbar. Eine Optimierung des Kommunikationsprozesses ist erforderlich, während gleichzeitig die Qualität und Konsistenz des Codes sichergestellt werden muss.

In diesem Kontext erscheint die Einführung der Codegenerierung als ein vielversprechender Ansatz. Codegenerierung aus OpenAPI-Spezifikationen, die eine API-Dokumentation als JSON-Datei anbietet, spart sowohl Zeit als auch Ressourcen, indem sie repetitive und fehleranfällige Aufgaben reduziert. Dieser Ansatz unterstützt die Einhaltung von Best Practices und Clean Code-Prinzipien und fördert auch einen einheitlichen Coding-Stil, was die Lesbarkeit und Wartbarkeit des Codes verbessert.

Die Zielsetzung dieses Kapitels ist es daher, ein Konzept für ein Codegenerierungstool zu erstellen. Als Basis wird dafür in Kapitel 4.4 der Ansatz aus Guenther Ruhes Checkliste (3.2.3) herangezogen.

4.1. Projektkontext und Notwendigkeit der Codegenerierung

Projekte im Bereich der Softwareentwicklung sind häufig von Verzögerungen und Budgetüberschreitungen betroffen (Kapitel 3.1.1) [30] [5]. Diese Probleme bestehen aus verschiedenen technischen und menschlichen Faktoren wie Wagner und Ruhe [48] darlegen. Oft sind die Faktoren Unterschätzung der Komplexität des Projekts, unvorhergesehene technische Schwierigkeiten oder ineffizienten Entwicklungsprozesse. Diese Herausforderungen reichen von Projektverzögerungen und Budgetüberschreitungen bis hin zu technical debt und der Notwendigkeit, Clean-Code-Refactoring zu betreiben. Spezifische Herausforderungen sind:

1. **Projektverzögerungen und Budgetüberschreitungen:** Wie durch die Untersuchungen von McKinsey & Company und der Harvard Business Review hervorgehoben, sind solche Probleme im Softwareentwicklungssektor weit verbreitet[30][5]. Durch Automatisierung der Erstellung von Boilerplate-Code und anderen wiederkehrenden Mustern kann Codegenerierung die Entwicklungszeiten verkürzen und dazu beitragen, Projektverzögerungen zu minimieren. Das führt unmittelbar zu einer besseren Einhaltung der Projektzeitpläne und Budgets.
2. **Technical debt:** Technical debts entstehen durch unzureichende Anfangsplanung, mangelnde Implementierung von Best Practices für sauberen Code und das Vorhandensein von "Quick Fixes"[15]. Automatisch generierter Code, der Best Practices folgt, kann dazu beitragen, die Anhäufung von technical debt zu vermeiden. Durch saubere, gut dokumentierte und wiederverwendbare Codekomponenten, wird die zukünftige Wartung vereinfacht und die Notwendigkeit für "Quick Fixes" reduziert.
3. **Code Qualität:** Die Entwicklung sauberer, wartbarer und erweiterbarer Codebasen wird immer wichtiger, um die langfristige Effizienz und Qualität von Softwareprodukten zu gewährleisten. 3.1.3[25]. Codegenerierungswerkzeuge, die Best Practices einhalten, können dabei unterstützen, einheitliche und hochwertige Codebasen zu entwickeln. Das führt dazu, dass der generierte Code den Clean-Code-Prinzipien entspricht und die Softwarequalität insgesamt verbessert wird.

Die Implementierung von Codegenerierung im Entwicklungsworkflow bietet eine direkte Antwort auf die in Kapitel 3.2 erarbeiteten Herausforderungen. Durch Fokus auf diese Strategie, kann nicht nur die Entwicklungseffizienz gesteigert werden, sondern auch Codequalität verbessert werden.

4.2. Rolle der Teamkommunikation

Die Grundlage für erfolgreiche Softwareprojekte ist eine effektive Kommunikation. Es ist wichtig, dass die Anforderungen und Spezifikationen präzise vermittelt werden, um Fehlentwicklungen und redundante Arbeitsschritte auszuschließen. Ineffizienzen auf diesem Gebiet verursachen nicht nur Zeitverluste, sondern auch eine deutliche Erhöhung der Kosten für Projekte. Aus Untersuchungen geht hervor, dass ein Mangel an Kommunikation zwischen den Teammitgliedern zu Missverständnissen führen kann, die die Ziele des Projekts in Gefahr bringen [48]. Die Studie [50] zeigt auch, dass ineffektive Kommunikationsprozesse, einen erheblichen Beitrag zu den Schwierigkeiten bei Entwicklungsprojekten leisten.

In Kapitel 3.2 wurden spezifische Kommunikationsprobleme innerhalb von Entwicklungsteams diskutiert. Diese Schwierigkeiten beinhalten u. a. eine unzureichende Vermittlung technischer Anforderungen, eine unzureichende Abstimmung zwischen den Teammitgliedern und eine ungenaue Auslegung von Spezifikationen. Die Wichtigkeit einer effektiven Kommunikation für den Erfolg von Softwareentwicklungsprojekten wird durch diese Schwierigkeiten gezeigt. Durch die Erstellung von Code würde ein Kommunikationsschritt eingespart bleiben.

4.2.1. Zeit- und Kostenersparnis durch Codegenerierung

Die Codegenerierung bewältigt diese Schwierigkeiten, indem sie es ermöglicht, aus vorgegebenen Spezifikationen automatisch Quellcode zu generieren. Dadurch wird der manuelle Arbeitsaufwand verringert, Fehlerquellen reduziert und eine einheitliche Anwendung von Entwicklungsstandards gewährleistet. Insbesondere die Erstellung von Code aus OpenAPI-Spezifikationen für RESTful APIs verdeutlicht die Effizienzsteigerung repetitiver und fehleranfälliger Aufgaben durch Automatisierung, was letztendlich zu Zeit- und Ausgabeneinsparungen führen kann.

1. **Kommunikation:** Codegenerierungswerkzeuge sind ein wirksames Mittel, um den Implementierungsprozess zu vereinfachen und die interne Kommunikation im Entwicklerteam zu verbessern. Diese Werkzeuge ermöglichen den Entwicklern, sich auf Designüberlegungen und das Lösen komplexer Herausforderungen zu konzentrieren, anstatt Unklarheiten bei den Spezifikationen auszuräumen, indem sie Softwareanforderungen genau definieren. Das hat zur Folge, dass die Effektivität und Wirksamkeit der Teamarbeit erhöht werden. Eine robuste Werkzeugunterstützung für die moderne Webentwicklung, insbesondere bei der Verwendung von Angular und REST-APIs, ist unverzichtbar. Die vorliegende Arbeit hebt die Bedeutung optimierter Workflows und die Vorteile automatisierter Codeerzeugung in der Softwareentwicklung hervor [48].

2. **Automatisierung:** Durch den Einsatz von Codegenerierungstools werden Anforderungen und Spezifikationen direkt in den erstellten Code übertragen. Dies führt zu einer Verringerung des Bedarfs an manuellen Auslegungen und einer genaueren Umsetzung der Anforderungen, wodurch das Risiko von Missverständnissen deutlich reduziert wird. Dies führt zu einer Verringerung des Bedarfs an manuellen Auslegungen und einer genaueren Umsetzung der Anforderungen, wodurch das Risiko von Missverständnissen deutlich reduziert wird. Die Untersuchung von Wagner und Ruhe 3.1.1 [48] zeigt, dass die Automatisierung in der Softwareentwicklung zu einer erheblichen Verringerung von Fehlern und Unstimmigkeiten führt und somit die Qualität und Effizienz von Projekten erhöht.
3. **Best Practices:** Die Generierung von Code hilft dabei, die Clean-Code-Prinzipien einzuhalten, da sie einen einheitlichen, gut strukturierten und lesbaren Code generiert. Martin [29] hebt hervor, wie wichtig es ist, sauberen Code zu erstellen, um Softwareprodukte langfristig zu warten und flexibel zu gestalten. Die Automatisierung dieses Vorgangs gewährleistet, dass der erstellte Code diesen Grundsätzen entspricht. Dies erleichtert die Wartung, Überprüfung und Erweiterung des Codes durch das Team und trägt zur Förderung eines gemeinsamen Verständnisses der Codegrundlage bei [25].

Codegenerierung zur Optimierung der Teamkommunikation ist eine unmittelbare Lösung für die Kommunikationsprobleme, die in Kapitel 3 identifiziert wurden. Durch die Automatisierung von wiederholenden und fehleranfälligen Aufgaben sowie Verringerung des manuellen Aufwands bei der Erstellung von Code können Entwicklerteams ihre Effizienz steigern, die Kosten senken und letztendlich Softwareprodukte von höherer Qualität bereitstellen.

4.3. Vorteile der Codegenerierung spezifisch für Angular

4.3.1. Auswahl von Angular

Angular wurde speziell für dieses Tool ausgewählt, da es ein umfangreiches Ökosystem, starke Typisierung und modulare Architektur bietet. Diese Eigenschaften vereinfachen die Entwicklung großer und komplexer Anwendungen [43]. Angular fördert die Einhaltung von Best Practices durch seine Struktur und Werkzeuge, die es Entwicklern erleichtern, qualitativ hochwertigen und wartbaren Code zu schreiben. Laut Stackoverflow bleibt Angular mit 17,46 % eines der Top-3 JavaScript-Frontend-Frameworks [42] 4.1. Im Jahr 2023 zeigt es Verbesserungen, vor allem eine Steigerung der Build-Geschwindigkeit um bis zu 87 %, überarbeitete Signalreaktivität und deferrable Views für eine verbesserte App-Performance, was es zu einer hervorragenden Option für die Integration von Codegenerierung macht [4].

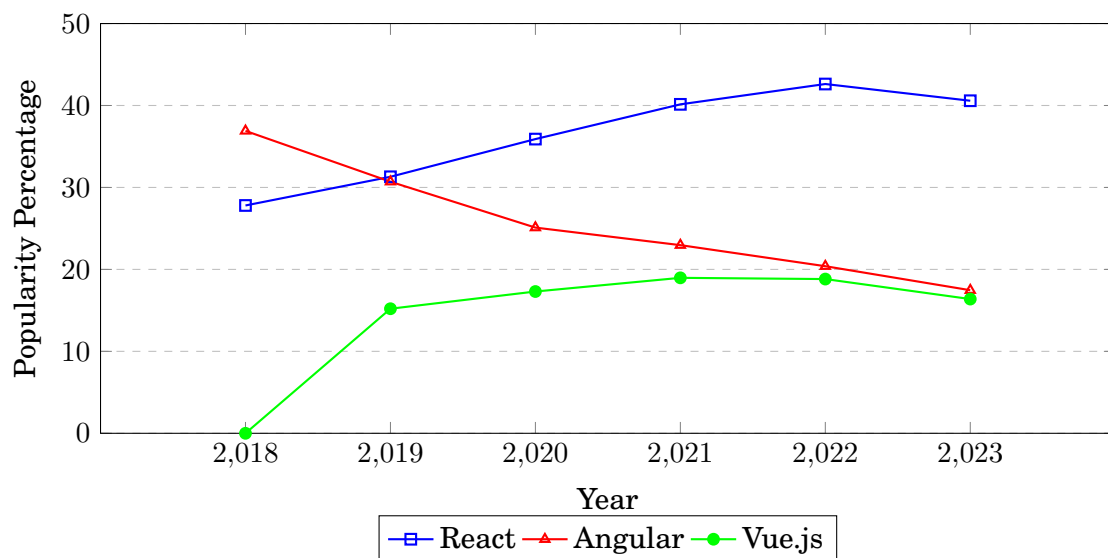


Abbildung 4.1.: Web Framework Popularity Over Time [42]

Die technischen Schwierigkeiten, die in Kapitel 3 festgestellt wurden, beinhalten die Komplexität des Anforderungsmanagements, die Integration neuer Technologien, die Einhaltung bewährter Verfahren und die Wartbarkeit. Die modulare Architektur von Angular und die starke Typisierung lösen unmittelbar diese Schwierigkeiten, indem sie die Verwaltung und Integration von Services vereinfachen [43]. Insbesondere für Angular kann die Codegenerierung diese Vorteile noch verstärken. Dies geschieht durch die automatisierte Erstellung von Boilerplate-Code, die Konsistenz und Einhaltung von Best Practices sowie die Erleichterung der Anforderungsverwaltung.

4.3.2. Best Practices, Clean Code und einheitlicher Coding-Stil

Für die langfristige Wartbarkeit und Skalierbarkeit von Softwareprojekten ist die Einhaltung von Best Practices und Anwendung von Clean Code Prinzipien entscheidend. Codegenerierungstools für Angular können einen konsistenten Coding-Stil im Team unterstützen, indem sie diese Prinzipien einhaltende, vordefinierte Codevorlagen verwenden. Dadurch wird nicht nur die Konsistenz gefördert, sondern auch die Lesbarkeit und Qualität des Codes verbessert.

Eine Vielzahl von Untersuchungen und Expertenmeinungen hebt die Wichtigkeit von Clean Code hervor. Neil Ernst verweist auf die kumulativen Kosten von technical debt die sich aus der Vernachlässigung von Clean Code Prinzipien ergeben können. Clean Code verbessert die Lesbarkeit und Wartbarkeit des Codes, wodurch technical debt reduziert werden [15].

John Ousterhouts „A Philosophy of Software Design“ bestätigt die Wichtigkeit von Clean Code und bewährten Verfahren für die Softwareentwicklung. Ousterhout behauptet, dass eine grundlegende Einfachheit und die Vermeidung von Komplexität für eine wirksame Softwaregestaltung von entscheidender Bedeutung sind [35]. Er hebt hervor, dass es essenziell ist, Software so zu gestalten, dass sie nicht bloß funktional, sondern auch leicht verständlich und wartbar ist. Entwickler können sicherstellen, dass ihre Anwendungen von Grund auf mit einem klaren Design und einer nachhaltigen Architektur ausgestattet sind, indem sie Ousterhouts Prinzipien auf Angular-Projekte anwenden und Codegenerierungstools verwenden.

Für Angular konzipierte Codegenerierungstools automatisieren die Anwendung dieser Prinzipien. Der CHAOS-Bericht und Coplien betonen, dass es für den Erfolg des Projekts von entscheidender Bedeutung ist, einen einheitlichen Coding-Stil einzuhalten [19][29]. Boilerplate-Code, der den Angular Style Guide und damit einen hohen Codequalitätsstandard erfüllt, wird von Tools wie Angular CLI erstellt.

Durch die automatische Anwendung von bewährten Verfahren und Prinzipien des Clean Code wird sichergestellt, dass die Qualität des Codes auch unter Zeitdruck erhalten bleibt. Martin beschreibt, dass die Einfachheit und Klarheit des Codes das Entwicklererlebnis und die Produktivität verbessern [29]. Diese Grundsätze werden in Angular-Projekten durch Codegenerierung angewendet, um die Qualität und Zukunftssicherheit des Codes zu gewährleisten, auch wenn rasches Ändern notwendig ist. Dies ist besonders bedeutend, da viele Entwickler nicht eigenständig Clean Code erstellen [25].

4.4. Konzept basierend auf Checkliste

4.4.1. ROI-Analyse

Die zugrundeliegende Annahme ist, dass die Entwicklung des Tools 200 Arbeitsstunden beansprucht und der Stundensatz symbolisch 10 Dollar beträgt. Daraus resultieren Gesamtentwicklungskosten von 2.000 Dollar.

Es folgt die Aufstellung hypothetischer Kostenkalkulationen. Die Entwicklungskosten bilden nur einen Teil der Gesamtausgaben. Für die laufende Wartung und für Aktualisierungen ist eine zusätzliche finanzielle Planung notwendig. Für diese Posten wird ein Jahresbetrag in Höhe von 20 % der ursprünglichen Entwicklungskosten veranschlagt was 400 Dollar entspricht.

Der Nutzen des Tools lässt sich durch Zeitersparnis quantifizieren: Ein Entwickler spart 50 Stunden pro Projekt, was bei einem erhöhten Stundensatz von 15 Dollar zu einer Kostenersparnis von 750 Dollar führt. Bei vier Projekten pro Jahr ergibt das eine jährliche Ersparnis von 3.000 Dollar.

Zur Berechnung des hypothetischen Return on Investment (ROI) dient folgende Formel: $ROI = (\text{Gesamtnutzen} - \text{Gesamtkosten}) / \text{Gesamtkosten}$. Unter Anwendung dieser Formel lässt sich der finanzielle Erfolg des Projekts abschätzen, indem der erzielte Nutzen ins Verhältnis zu den aufgewendeten Gesamtkosten gesetzt wird.

$$ROI = \left(\frac{(\$3,000 + \$400) - (\$2,000 + \$400)}{\$2,000 + \$400} \right) \times 100\% = 8.33\%$$

Es ist anzumerken, dass, obwohl die ROI-Berechnung einen gewissen finanziellen Nutzen aufzeigt, dieser im Rahmen dieser Arbeit als vernachlässigbar betrachtet wird.

4.4.2. Modeling and Problem Formulation

Die vorangegangenen Kapitel 1.3 und 3 haben gezeigt, wie wichtig es ist, die Entwicklungsabläufe für die Webentwicklung zu optimieren. Darüber konnte festgestellt werden, dass die manuelle Bindung von Client- und Serverkomponenten innerhalb von RESTful-APIs immer wieder mit Schwierigkeiten verbunden ist.

Das bereitgestellte Anwendungsfalldiagramm (siehe 4.2) veranschaulicht die Interaktion eines Entwicklers mit einem Codegenerierungswerkzeug. Es zeigt den systematischen Ablauf, der vom Entwickler initiiert wird und zur Generierung von Client-Klassen über NPM-Befehle unter Verwendung von OpenAPI-JSON-Dateien aus der Swagger-Dokumentation führt.

Nach erneuter Betrachtung der ursprünglichen Problemstellung kann eine konkrete Problemformulierung abgeleitet werden, die die Variablen - wie die als Eingabe vorbereiteten JSON-Dateien - und die Randbedingungen, einschließlich der von TypeScript für Angular-Anwendungen geforderten Struktur und Syntax, abbildet.

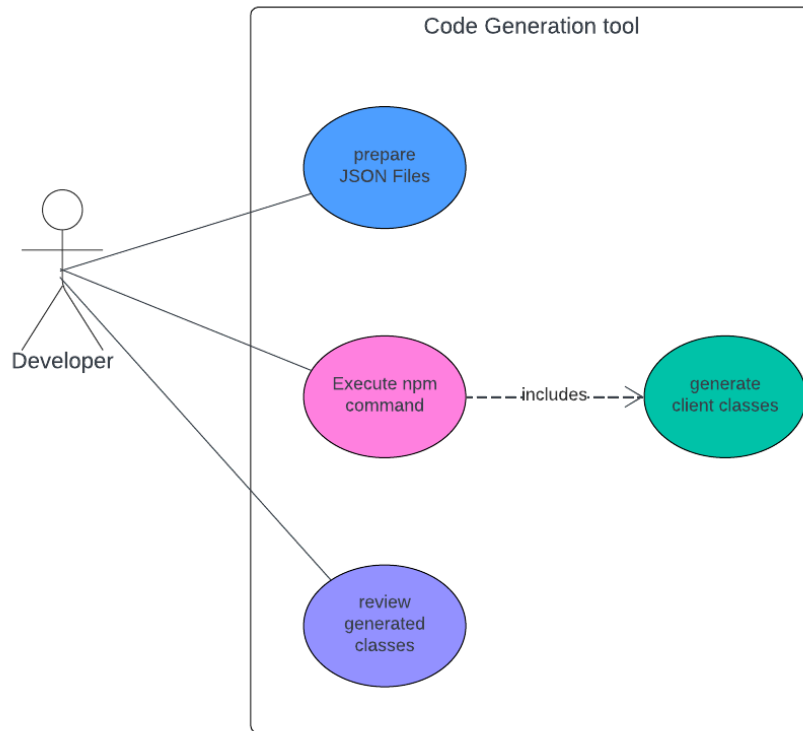


Abbildung 4.2.: Use case diagram

- Vorbereiten von JSON-Dateien: Der Entwickler stellt sicher, dass die OpenAPI-JSON-Datei die neuesten API-Spezifikationen enthält. Ausführen des NPM-Befehls: Mithilfe des NPM Befehls führt der Entwickler das Tool aus.
- Generierung von Client-Klassen: Das Codegenerierungstool verarbeitet das eingegebene JSON und gibt TypeScript-Klassen aus, die mit der Angular-Architektur übereinstimmen.
- Überprüfung der generierten Klassen: Abschließend überprüft der Entwickler die generierten Klassen auf Konsistenz und Einhaltung vordefinierter Standards.

Dieses Modell ist die Grundlage für optimierungen der Softwarequalität und des Workflows durch Codegenerierung.

4.4.3. Solution Design

Die Phase des Solution Design befasst sich mit den komplexen Fragen "Was?" und "Wie viel ist genug?", die in der Problemstellung in Kapitel 1.3 umrissen wurden. Aufbauend auf der umfassenden Problemanalyse, die in Kapitel 3 durchgeführt wurde, orientiert sich dieser Abschnitt an den gewünschten Ergebnissen und dem Umfang der Lösung, die in Kapitel 4.4.2 festgelegt wurden.

Die zuvor hervorgehobenen Probleme konzentrieren sich auf die Ineffizienz der Arbeitsabläufe und die Fehleranfälligkeit des Codes. Diese werden von der Notwendigkeit eines schlanken Ansatzes umrahmt, der die Komplexität verringert und die Wartbarkeit fördert. Bei der Frage, "Wie viel ist genug?" geht es darum, dass das Minimum Viable Product (MVP) identifiziert werden muss, das alle Kernfunktionen enthält, ohne die Lösung zu überfrachten.

Die Untersuchung von Alternativen beinhaltet eine gründliche Prüfung potenzieller Werkzeuge und Methoden. Es wird eine Ausgewogenheit innovativer Fähigkeiten und bewährter Zuverlässigkeit angestrebt. Die Untersuchung orientiert sich an den detaillierten Erkenntnissen aus Kapitel 3 und stellt sicher, dass der gewählte Weg mit den festgelegten Kriterien übereinstimmt.

Die Auswahl von OpenAPI für das Design von RESTful-API-Schnittstellen in Kombination mit TypeScript für Angular-Anwendungen stützt sich auf Erkenntnissen aus der Modellierung und Problemformulierung in Kapitel 4.4.2. Die OpenAPI-Spezifikation bietet ein Framework für die Definition von Schnittstellen, das durch die starke Typisierung von TypeScript für die Komponentenarchitektur von Angular erweitert wird.

Der Entwickler beginnt mit der Vorbereitung von OpenAPI-JSON-Dateien, die die Swagger-Dokumentation widerspiegeln, und führt NPM-Befehle aus, um TypeScript-Dateien für Angular zu generieren - Schritte, die für die nachfolgende Codegenerierung entscheidend sind.

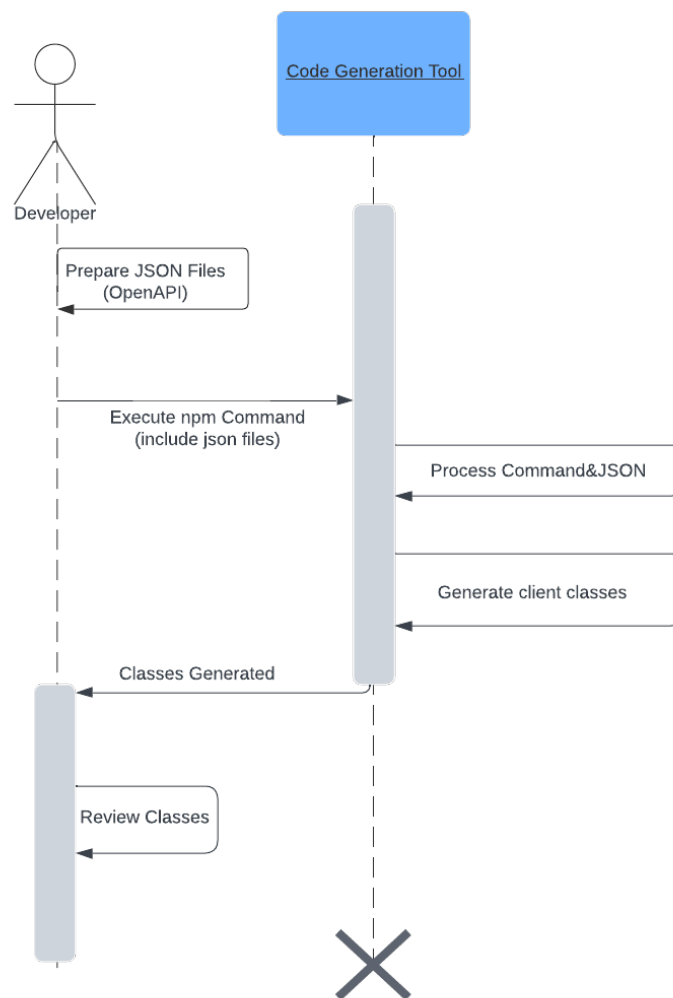


Abbildung 4.3.: Sequence diagram

Das bereitgestellte Sequenzdiagramm bietet eine Visualisierung der Interaktionen zwischen dem Entwickler und dem Codegenerierungswerkzeug während des TypeScript-Dateigenerierungsprozesses für Angular-Anwendungen.

- Der Entwickler beginnt mit der Vorbereitung der OpenAPI-JSON-Dateien, die die formalen Beschreibungen der RESTful-API darstellen. Diese JSON-Dateien werden aus der Swagger-Dokumentation abgeleitet, in der die API-Schnittstellen definiert sind.
- Nach der Vorbereitung der JSON-Dateien führt der Entwickler den erforderlichen NPM-Befehl aus. Dieser Befehl ruft das Codegenerierungstool auf.

- Das Codegenerierungstool empfängt den NPM-Befehl und die zugehörigen JSON-Dateien. Es verarbeitet sie und interpretiert die API-Strukturen und die erwarteten Ausgaben.
- Sobald die Verarbeitung abgeschlossen ist, generiert das Tool Client-Klassen. Bei diesen Klassen handelt es sich im Wesentlichen um TypeScript-Dateien für Angular, die sich an der Struktur und den Anforderungen orientieren, die in den OpenAPI-JSON-Dateien angegeben sind.
- Das Ergebnis der Ausführung des Tools ist eine Reihe von generierten Klassen. Diese Klassen bilden das Gerüst für die Angular-Anwendung und ermöglichen eine starke Typisierung und die Einhaltung der definierten API-Spezifikationen.
- Der Entwickler überprüft die generierten Klassen, um sicherzustellen, dass sie den Anforderungen und Standards des Projekts entsprechen. Dieser Schritt ist besonders relevant, da er die Qualität des Codes prüft.

Diese Methoden werden auf der Grundlage der ROI-Analyse in Kapitel 4.4.1 ausgewählt, um sicherzustellen, dass die Lösung sowohl kosteneffizient als auch wirkungsvoll ist.

4.4.4. Data Collection

Durch den Rahmen von Günther Ruhe betrachten wir, wie empirische Studien zur Verbesserung der Menge und Qualität der Daten beitragen können. Anstelle einer breiten empirischen Datengrundlage beschränkt sich die Untersuchung auf spezifische Einsichten, die aus dem Interview mit Tech Lead Paul Schnabel gewonnen wurden. Dieser Ansatz stellt eine Anpassung und Vereinfachung der von Ruhe vorgeschlagenen Checkliste dar, da die Menge an Daten in diesem Kontext begrenzt ist.

Da das Hauptaugenmerk auf dem Interview liegt, werden die darin enthaltenen Informationen als Basis für die Datensammlung genutzt. Zunächst werden die Aussagen Schnabels hinsichtlich ihrer Implikationen für das Konzept analysiert. Daraus werden spezifische Datenpunkte abgeleitet. Diese Vorgehensweise ermöglicht es, auch ohne eine umfangreiche Datenbasis Schlüsse zu ziehen und die Optimierungsstrategie auf einer soliden Informationsgrundlage aufzubauen.

1. **Effizienzsteigerung:** Eine der Hauptvorteile der Codegenerierung, wie sie aus dem Interview hervorgeht, ist die Steigerung der Entwicklungseffizienz. Automatisierte Prozesse reduzieren den manuellen Codierungsaufwand, beschleunigen die Entwicklungszyklen und ermöglichen es Teams, schneller auf Anforderungsänderungen zu reagieren.

2. **Qualitätsverbesserung:** Durch die Standardisierung von Codefragmenten und -mustern trägt die Codegenerierung dazu bei, Konsistenz und Qualität des Codes zu verbessern. Tests und Validierungen, die in den Generierungsprozess integriert sind, helfen bei der Identifizierung und Korrektur von Fehlern.
3. **Anpassungsfähigkeit:** In einem sich schnell ändernden technologischen Umfeld ermöglicht die Codegenerierung eine bessere Anpassungsfähigkeit an neue Anforderungen. Codegeneration kann genutzt werden, um schnell Prototypen zu erstellen.

Im Interview mit Paul Schnabel werden spezifische Szenarien und Herausforderungen angesprochen, die die Einführung von Codegenerierungstechniken als Lösungsansatz nahelegen. Durch die Analyse dieser Aussagen wird klar, wie Codegenerierung nicht nur technische, sondern auch geschäftliche Ziele unterstützt. Es wird bspw. die Notwendigkeit, Entwicklungsprozesse zu beschleunigen, um mit Marktanforderungen Schritt zu halten betont. Dabei ist Codegenerierung ein Werkzeug, um diese Agilität zu erreichen.

4.4.5. Optimization

Die Optimization befasst sich mit der Spezifizierung von Parametern um die Robustheit der Lösung zu bewerten. Robuster Code zeichnet sich durch seine Widerstandsfähigkeit gegenüber Änderungen in den Eingabeparametern und Anforderungen aus und gewährleistet die Lieferung verlässlicher Ergebnisse unter diversen Bedingungen.

Dafür werden die genauen Inhalte des Outputs festgelegt:

Struktur der Klassen

Die Klassenstruktur eines durch ein Codegenerierungs-Tool erzeugten Angular-Moduls setzt sich aus typisierten Attributen zusammen, die eine direkte Entsprechung zu den Datenmodellen haben.

Konstruktoren und Dependency Injection

Jede Klasse wird mit einem Konstruktor versehen, der die Initialisierung erleichtert und dabei den Grundsätzen der Dependency Injection folgt.

Methoden und Klassenverhalten

Methoden innerhalb der Klassen steuern das Verhalten der Komponenten und sorgen dafür, dass die Logik klar definiert und wiederverwendbar ist. Sie ermöglichen es, Funktionen zu isolieren und separat zu testen.

Dekoratoren und ihre Verwendung

Dekoratoren wie `@Injectable` erlauben es, Klassen als Services zu definieren, die im gesamten Modul verfügbar gemacht werden können.

Services und Observable

Die Integration von HTTP-Services in Klassen mittels HTTP-Client ermöglicht die Kommunikation mit Backend-Systemen über RESTful APIs. Diese Services verwenden HTTP-Anfragen, um Daten zu lesen, erstellen, aktualisieren oder löschen.

Observable und asynchrone Operationen

Die Rückgabe von Observables aus Service-Methoden ermöglicht die Handhabung von asynchronen Datenströmen.

Modelle und Modularisierung

Interfaces und Klassen bilden die Grundlage für die Definition und das Typing der Datenmodelle, die die Anwendungslogik verwenden wird.

NgModule und Projektorganisation

Jede Klasse wird in einem NgModule organisiert, das die notwendigen Angular-spezifischen Deklarationen und Provider zusammenfasst.

Dokumentation und Fehlerbehandlung

Jede Funktion hat ein automatisch generiertes Errorhandling, durch die Implementierung einer Middleware die HTTP Statuscodes überprüft. Am Anfang jeder Datei wird der Hinweis stehen, dass diese automatisch generiert wurde.

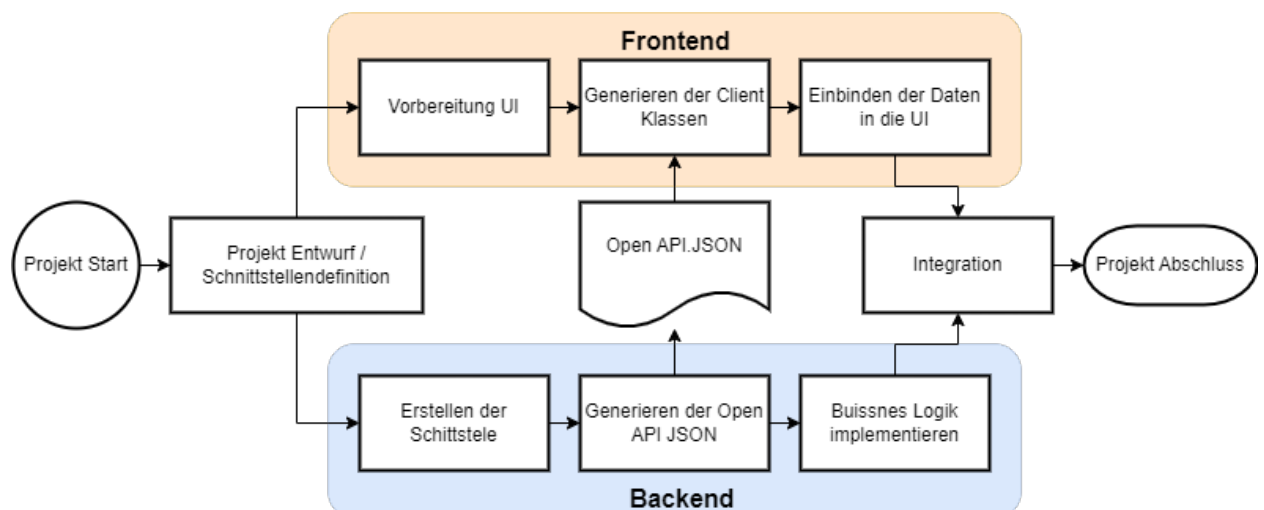


Abbildung 4.4.: Vereinfachter Workflow mit Client Generierungstool

Im Kontext dieses Entwicklungsprozesses, wie im beigefügten Flussdiagramm (4.4) dargestellt, dient das Angular Codegen-Tool der Erstellung von Klassenstrukturen für das Frontend, die nahtlos mit den im Backend definierten APIs interagieren. Das Flussdiagramm veranschaulicht den gesamten Ablauf von der Projektinitiierung bis hin zum Projektabschluss und zeigt auf, wo die Generierung der Klassen und Services in den Prozess der Softwareentwicklung passt.

4.4.6. Validation

Die Validation des entwickelten Optimierungskonzepts ist ein entscheidender Schritt, um die Eignung und Effizienz des Ansatzes im Vergleich zu bestehenden Lösungen zu bewerten. Dieses Kapitel konzentriert sich auf die Evaluierung der generierten Lösung, indem sie dem, im Interview mit Paul Schnabel 3.2.2 erwähnten Code-Generierungstool, gegenübergestellt wird. Der Beispielcode findet sich in Anhang B Code.

Der Fokus des Vergleichs liegt auf der Architektur, wobei das existierende Tool alle Klassen und Interfaces in einer einzigen Datei zusammenführt. Im Gegensatz dazu zielt die geplante Architektur darauf ab, jede Klasse und jedes Interface in einer eigenen Datei zu organisieren. Dieser Ansatz bietet mehrere Vorteile in Bezug auf Wartbarkeit, Skalierbarkeit und Kollaboration.

Diese Struktur erleichtert das Auffinden spezifischer Codeabschnitte und fördert eine sauberere Codebasis. Im Vergleich dazu kann das Zusammenführen aller Klassen und Interfaces in einer einzigen Datei die Wartbarkeit erschweren, da Änderungen an einer Stelle unbeabsichtigte Effekte in anderen Teilen des Codes haben können.

Die bisherige Validierung und der Vergleich zwischen der geplanten Architektur und dem existierenden Code-Generierungstool, wie im Interview mit Paul Schnabel 3.2.2 erörtert und durch den Beispielcode in Anhang B dargestellt, zeigen das Potential des neuen Ansatzes. Ein vollständiger Vergleich ist allerdings erst möglich, nach erfolgreicher Implementierung des Tools.

Die Punkte Implementierung und Evaluation von Guenther Ruhes Checkliste (3.2.3), folgen in den Kapiteln 5 und 7.

5. Implementierung

In diesem Kapitel wird die konkrete Implementierung des im vorherigen Kapitel entwickelten Lösungskonzepts erläutert, das auf Guenther Ruhes Checkliste aufbaut. Hierbei liegt der Fokus auf der praktischen Umsetzung der entworfenen Software-Architektur, unter Berücksichtigung der in der Softwareentwicklung etablierten Prinzipien und Methoden. Dabei wird auf die Implementierung von Diensten und Schnittstellen zur Interaktion mit einer OpenAPI-Spezifikation eingegangen.

5.1. Entwicklungsansatz

Der Entwicklungsansatz orientiert sich an den Prinzipien des prozeduralen Programmierens und beachtet dabei Prinzipien der Softwareentwicklung. Jede Funktion hat eine spezifische Aufgabe und wird versucht möglichst kurz gehalten zu werden [29], ohne dabei einen dogmatischen Ansatz einer Maximallänge zu nutzen [35]. Dabei wurde das "Don't repeat yourself" Prinzip [29] eingehalten.

5.1.1. Kernfunktionen

Die Implementierung umfasst mehrere Schlüsselkomponenten, die im Folgenden kurz beschrieben werden:

1. **Generierung von Angular Services:**

Die Funktion `generateAngularServices` analysiert die OpenAPI-Spezifikation und generiert Angular Services, die den in der Spezifikation definierten Endpunkten entsprechen. Jeder Service ermöglicht die Kommunikation mit einem spezifischen Teil der API.

2. **Erstellung von Interfaces:**

Mit `generateInterfacesFromSchemas` wird aus den in der OpenAPI-Spezifikation definierten Schemata TypeScript-Interfaces generiert. Dies fördert die Typsicherheit und erleichtert die Entwicklung.

3. Dynamische Service-Generierung pro Tag:

Durch die Zuordnung von API-Endpunkten zu spezifischen Tags wird mit `generateServiceForTag` eine genaue Organisation der Services für eine höhere Wartbarkeit des Codes ermöglicht.

4. Automatische Fehlerbehandlung:

Die Implementierung einer Middleware (`generateMiddleWare`) zur zentralisierten Fehlerbehandlung ermöglicht einheitliche Reaktionen auf Fehlerzustände und vereinfacht das Fehlermanagement im gesamten Projekt.

5. Automatische Methodengenerierung:

Die Funktion `generateServiceMethods` spielt eine zentrale Rolle in der Erstellung von Angular Services. Sie nimmt eine Liste von API-Pfaden (`PathOperation`) entgegen, die jeweils einen Endpunkt der OpenAPI-Spezifikation repräsentieren, und generiert für jeden Pfad eine entsprechende Methode im Angular Service.

5.1.2. Modularität und Separation of Concerns

Die Aufteilung des Codes in spezifische Funktionen für verschiedene Aufgaben unterstützt eine klare Trennung von Belangen (Separation of Concerns). Dies verbessert nicht nur Lesbarkeit und Wartbarkeit des Codes, sondern erleichtert auch zukünftige Erweiterung und Anpassung der Anwendung [35][29]. Funktionen wie `generateServiceMethods` (siehe 5.1) kapseln komplexe Logik hinter relativ einfachen Schnittstellen. Dadurch folgen sie dem Prinzip der tiefen Modularität [35].

```
1 function generateAngularServices(spec: OpenApiSpec) {
2   const pathsByTag: PathsByTag = {}
3   Object.keys(spec.paths).forEach((path) => {
4     Object.keys(spec.paths[path]).forEach((method) => {
5       const operation = spec.paths[path][method]
6       const tags = operation.tags
7       tags.forEach((tag) => {
8         pathsByTag[tag] = pathsByTag[tag] || []
9         pathsByTag[tag].push({ path, method, operation })
10      })
11    })
12  })
13
14  Object.keys(pathsByTag).forEach((tag) => {
15    return generateServiceForTag(tag, pathsByTag[tag],
16                                spec.components.schemas)
17  })
18 }
```

Quelltext 5.1: Generierung von Angular Services

5.1.3. Dokumentation und Code-Style

Die Implementierung verfolgt den Grundsatz, dass gut geschriebener Code sich selbst erklärt und somit umfangreiche Kommentierungen entbehrlich macht [29][35]. Code sollte in einer Art und Weise geschrieben werden, die seine Funktion und Absicht klar und deutlich macht. Kommentare werden daher sparsam eingesetzt und beschränken sich auf Fälle, in denen das Verhalten des Codes nicht offensichtlich ist oder spezifische Entscheidungsprozesse hinter der Implementierung erläutert werden müssen, wie 5.2 "Funktion mit Sideeffect" dargestellt ist.

```
1 export async function setUpAPIService(  
2   openApiSpecPath: string,  
3   usePrettier: boolean,  
4 ) {  
5   const openApiSpecContents = fs.readFileSync(openApiSpecPath,  
6     'utf8')  
7   const openApiSpec = JSON.parse(openApiSpecContents)  
8   fs.mkdirSync('services', { recursive: true })  
9   //side effect, that is used for cleaning up files  
10  usePrettierGlobal = usePrettier  
11  generateAngularServices(openApiSpec)  
12  generateMiddleWare()  
13 }
```

Quelltext 5.2: Funktion mit Sideeffect

Um den Code-Stil konsistent zu halten und die Lesbarkeit weiter zu verbessern, werden zusätzlich die Tools `eslint`¹ und `prettier`² zur Formatierung des Codes eingesetzt. Das sorgt dafür, dass die gesamte Codebase einem einheitlichen Formatierungsstandard folgt, was die Wartbarkeit und Verständlichkeit des Codes fördert.

5.2. Generierung des Angular-Codes

Die Entwicklung einer modernen Softwarearchitektur erfordert ein tiefes Verständnis von Design-Prinzipien und Best Practices. Dieses Kapitel konzentriert sich auf die praktische Anwendung dieser Prinzipien bei der Generierung von Angular Code, um eine effiziente, wartbare und skalierbare Frontend-Architektur zu erstellen.

¹"ESLint is an open-source Javascript linting utility"[16]

²"Prettier is an opinionated code formatter"[49]

5.2.1. Modularität durch Serviceorientierte Architektur

Indem jeder Angular-Service in einer eigenen Datei implementiert wird, wird Modularität und Unabhängigkeit der Komponenten gefördert. Dieses Vorgehen erlaubt es, Änderungen an einem Service vorzunehmen, ohne andere Teile der Anwendung zu beeinträchtigen. Es entspricht dem Prinzip der Trennung von Belangen und der modularen Architektur [27][28][35](siehe 4.4.5) geplant war. Es spiegelt das Prinzip der Tiefe statt Breite wider [35] und das Single Responsibility Principle [29].

Die Definition von Interfaces in einer zentralen Datei, die von allen Services importiert wird, fördert die Wiederverwendung von Code und stellt sicher, dass alle Services nach denselben Verträgen arbeiten. Dies unterstützt das Prinzip der Entkopplung und fördert eine konsistente Kommunikation zwischen den Teilen Ihrer Anwendung. Dies entspricht dem Interface Segregation Principle (ISP) und der Dependency Rule [28][35].

Indem für jeden Service eine dedizierte Klasse mit einem spezifischen Zweck erstellt wird, wird dem Prinzip "Tiefe statt Breite" gefolgt, was bedeutet, dass die Services eine starke Funktionalität in einem gut definierten Bereich bieten, anstatt viele nicht zusammenhängende Funktionen zu haben [35][29]. Jede Klasse wird mit einem Konstruktor versehen, der die Initialisierung erleichtert und dabei den Grundsätzen der Dependency Injection folgt, wie in 4.4.5 "Konstruktoren und Dependency Injection" vorgesehen war (Siehe 5.3).

```
1 function generateServiceClass(className: string, methods: string)
  {
2   const angularService = SERVICE_TEMPLATE.replace(
3     '{{className}}',
4     className,
5   ).replace('{{methods}}', methods)
6   return angularService
7 }
```

Quelltext 5.3: Erstellung des Konstruktors

Dieser Ansatz entspricht auch den Angular Best Practices aus [43].

5.2.2. Code-Struktur und -Qualität

Die Betonung der Codestruktur und -qualität ist für die Entwicklung von Software, die effizient und fehlerfrei ist, unverzichtbar. Der hier vorgestellte Entwicklungsansatz umfasst die Verwendung von aussagekräftigen Namen für Variablen, Klassen, Funktionen und Interfaces, durch die Vorgaben des Backends. Das soll eine klare und verständliche Codebasis fördern [29][35][10].

Eine zentrale Middleware für das Errorhandling ist unerlässlich für gutes Errorhandling. Diese Strategie nutzt die Empfehlungen für die effektive Behandlung von Cross-Cutting Concerns, die die Robustheit und Wartbarkeit der Anwendung verbessert [35]. Dies passiert durch die Funktion `generateMiddleWare` basierend auf Kapitel 4.4.5 "Dokumentation und Fehlerbehandlung".

5.3. Implementierung des CLI-Tools

Um die User Experience zu optimieren, wurde das Command Line Interface (CLI)-Tool mit einer interaktiven Oberfläche entwickelt. Dieses Interface bietet dem Benutzer während des gesamten Generierungsprozesses Rückmeldungen. Beim Start begrüßt das Tool den Benutzer mit einer blau gefärbten Willkommensnachricht, die mit `chalk.blue` erstellt wurde, um ein benutzerfreundliches Interface zu schaffen. Durch die Verwendung von `inquirer` wird der Benutzer aufgefordert, den Dateipfad anzugeben. Falls der Pfad nicht existiert, gibt das Tool eine klare und farblich hervorgehobene Fehlermeldung aus, die mit `chalk` rot gefärbt ist, um auf ein Problem hinzuweisen. Ein Spinner, implementiert durch `nanospinner`, visualisiert den Fortschritt des Prozesses und zeigt dem Benutzer, dass die Eingaben bearbeitet werden.

```
1  const spinner = createSpinner('Processing...\n').spin()
```

Quelltext 5.4: Implementierung des spinners

Nach erfolgreichem Abschluss der Dateigenerierung verkündet das Tool den Erfolg mit einer animierten Botschaft, generiert durch `figlet`, und gibt dem Benutzer die Rückmeldung, dass alle Angular-Dateien erfolgreich im Verzeichnis `services` gespeichert wurden. Diese Meldung wird in einem Regenbogenfarbschema durch `chalkRainbow` dargestellt.

```
1  await setUpAPIService(filePath, usePrettier)
2  const data = figlet.textSync('Success!')
3  spinner.success({ text: chalkRainbow(data) })
4  console.log(
5      chalk.blue(
6          "Your Angular files have been stored in services
           Directory! Let's go!",
7      ),
8  )
```

Quelltext 5.5: Rückmeldung für den User

Es bestätigt nicht nur, dass die Eingaben angenommen und verarbeitet werden, sondern vermittelt auch den Erfolg der Operation, was für die Benutzerzufriedenheit wesentlich ist. Eine Darstellung des Tools findet sich in 5.1.

6. Inbetriebnahme

6.1. Konfiguration der `package.json` und `tsconfig.json`

Zur Inbetriebnahme der Node.js Anwendung mussten die `package.json` und `tsconfig.json` Dateien für die korrekte Erstellung und Ausführung des Heigen¹-Pakets konfiguriert werden. In der `package.json` werden verschiedene Schlüsseleigenschaften definiert:

- `"main": "dist/index.js"` gibt den Entrypoint des Pakets an, nachdem es gebaut wurde.
- `"types": "dist/index.d.ts"` definiert den Speicherort der kompilierten TypeScript-Files.
- `"type": "module"` signalisiert, dass es sich um ein ES-Modul handelt.
- Unter `"bin"` wird `{"NgHeiGen": "dist/index.js"}` eingetragen, was ermöglicht, das Paket als Befehlszeilenwerkzeug zu verwenden.

In den `scripts`-Eigenschaften werden zwei Skripte definiert:

- `"build": "tsc"` verwendet den TypeScript Compiler, um den Code zu kompilieren.
- `"start": "node dist/index.js"` dient dazu, das Projekt direkt zu starten.

Die `tsconfig.json` beinhaltet den Eintrag `"outDir": "./dist"`, welcher angibt, dass alle kompilierten Dateien im Verzeichnis `dist` abgelegt werden.

¹Heigen ist der Name des NPM-Pakets

6.2. Erstellung der `.npmrc` Datei

Die `.npmrc` Datei wird benötigt, um die Authentifizierung gegenüber einem privaten NPM-Registry-Server sicherzustellen. Diese Datei enthält unter anderem die URL des NPM-Registries und Authentifizierungsinformationen in der Form von Basis-64 kodierten Tokens. Die wichtigsten Konfigurationen sind:

```
registry=https://[NPM_ARTIFACT_LINK]
//[NPM_ARTIFACT_LINK]:username=HEITEC
//[NPM_ARTIFACT_LINK]:_password=BASE64_ENCODED_PERSONAL_ACCESS_TOKEN
//[NPM_ARTIFACT_LINK]:email=[email]
//[NPM_ARTIFACT_LINK]:username=HEITEC
//[NPM_ARTIFACT_LINK]:_password=BASE64_ENCODED_PERSONAL_ACCESS_TOKEN
//[NPM_ARTIFACT_LINK]:email=[email]
always-auth=true
```

6.3. Build und Deployment

Um das Paket zu bauen und bereitzustellen, wurden die folgenden Befehle verwendet:

```
npm run build
npm publish
```

Die Installation und Nutzung des Pakets in einem Projekt erfordert eine `.npmrc` Datei mit einem gültigen, basis-64 kodierten persönlichen Zugangscode. Nach der Installation durch `npm install heigen@1.0.3` kann das Paket mit `npx heigen` ausgeführt werden.

7. Evaluierung

7.1. Einführung

In diesem Kapitel wird die Effektivität des Codegenerierungstools, in bezugnahme auf die spezifischen Ziele, die zu Beginn der Arbeit festgelegt wurden (1.4) evaluiert. Eine nicht repräsentative Umfrage unter den Mitarbeitern der Entwicklungsabteilung, unterstützt die qualitative Analyse.

7.2. Ziel 1: Definition und Implementierung des Entwicklungsworkflows

7.2.1. Erwartetes Ergebnis

Das erwartete Ergebnis war eine effizientere Gestaltung der Entwicklungsprozesse durch die Einführung von automatisierten Codegenerierungen.

7.2.2. Erreichte Ergebnisse

Durch die Implementierung der Codegenerierungstools konnte die Zeit für Standardaufgaben signifikant reduziert werden. Der Frontend-Entwickler kann wichtige Teile seines Codes generieren lassen, nachdem das Rest-API Design fertig gestellt ist.

7.2.3. Diskrepanzen und deren Ursachen

Der Workflow wurde für einen sehr spezifischen Anwendungsfall optimiert, nämlich Software, die im Frontend Angular nutzt und ein Backend mit einer OpenAPI Spezifikation.

7.3. Ziel 2: Reduktion von Boilerplate-Code

7.3.1. Erwartetes Ergebnis

Eine deutliche Reduktion des manuell zu schreibenden Codes wurde erwartet.

7.3.2. Erreichte Ergebnisse

Der Einsatz von Codegenerierung führte zu einer Reduktion von Boilerplate-Codes.

7.3.3. Diskrepanzen und deren Ursachen

Die Komplexität einiger spezifischer Anwendungen erforderte manuelle Anpassungen, die die vollständige Eliminierung von Boilerplate-Code verhinderten.

7.4. Ziel 3: Steigerung der Effizienz, Qualität und Wartbarkeit der Softwareentwicklung

7.4.1. Erwartetes Ergebnis

Eine Steigerung der Effizienz und Qualität der Softwareentwicklung durch die Straffung von Arbeitsabläufen und die Reduzierung von Boilerplate-Code wurde erwartet. Die Optimierung der Prozesse sollte auch zu einer besseren Wartbarkeit und Zukunftssicherheit der Softwareprodukte führen, indem sie langfristig Zeit und Ressourcen spart.

7.4.2. Erreichte Ergebnisse

Durch den optimierten Workflow und die Automatisierung wurden die Entwicklungszeiten verkürzt, was dem Team ermöglichte, sich auf die Verbesserung der Qualität und andere kritische Aspekte der Entwicklung zu konzentrieren.

7.4.3. Diskrepanzen und deren Ursachen

Siehe Ergebnis 2 (7.3): Die Effizienz wurde für einen sehr spezifischen Anwendungsfall optimiert.

7.5. Ziel 4: Sicherstellung der Erweiterbarkeit

7.5.1. Erwartetes Ergebnis

Die entwickelten Systeme bleiben flexibel und lassen sich leicht an neue Anforderungen anpassen.

7.5.2. Erreichte Ergebnisse

Die entwickelten Systeme bleiben trotz der Optimierungen und der Einführung automatisierter Prozesse flexibel und anpassbar. Dies zeigt sich darin, dass der Code einfach manuell veränderbar oder erweiterbar ist, was die schnelle Anpassung an neue Anforderungen oder die Integration neuer Funktionen erleichtert.

7.5.3. Diskrepanzen und deren Ursachen

Je nach Komplexität der Anwendung muss der Code manuell erweitert werden.

7.6. Vergleich mit dem alten Codegenerierungstool

Die Vergleiche zeigen, dass das neue Tool eine klarere Trennung von Klassen und Interfaces (siehe B.1) ermöglicht und verbessertes Errorhandling aufweist, was zu einer erhöhten Wartbarkeit beiträgt. Der Code ist, obwohl weniger Boilerplate-Code generiert wird, lesbarer und vermeidet die Verwendung von `any`-Types, was zu mehr Type-Safety führt (vergleiche B.2).

7.7. Umfrage

7.7.1. Kontext der Umfrage

Die durchgeführte Umfrage umfasst Antworten von acht Mitarbeitern der Softwareentwicklungsabteilung, die speziell mit Webentwicklung vertraut sind. Die Umfrage ist nicht repräsentativ für die gesamte Branche, da die Teilnehmerzahl begrenzt und auf einen spezifischen Bereich der Softwareentwicklung konzentriert ist. Die geringe Anzahl von Antworten ist teilweise darauf zurückzuführen, dass nicht mehr Mitarbeiter in der Abteilung mit Webentwicklung beschäftigt sind. Diese Faktoren limitieren die Generalisierbarkeit der Ergebnisse auf andere Kontexte oder größere Populationen.

Die vollständige Umfrage findet sich unter C Anhang Umfrage

7.7.2. Bewertungsübersicht

Die Umfrageergebnisse sind überwiegend positiv, mit Bewertungen, die meist zwischen 4 und 5 auf einer Skala von 1 (gar nicht zufrieden) bis 5 (sehr zufrieden) liegen. Die Ergebnisse im Detail sind wie folgt:

- **Wie zufrieden sind Sie insgesamt mit dem Codegenerierungstool?** 4/5
- **Wie leicht fiel es Ihnen, das Tool zu erlernen und zu verwenden?** 4.3/5
- **Wie bewerten Sie das Design und die Benutzeroberfläche des Tools?** 4/5
- **Inwiefern erfüllt das Tool Ihre Erwartungen bezüglich der Codegenerierung?** 4.6/5
- **Wie effektiv hilft Ihnen das Tool, Zeit bei der Entwicklung zu sparen?** 5/5
- **Wie bewerten Sie die Geschwindigkeit und Reaktionsfähigkeit des Tools?** 4.6/5
- **Wie oft sind Sie auf Leistungsprobleme gestoßen?** 5/5
- **Wie häufig treten Fehler oder Bugs während der Nutzung des Tools auf?** 4.3/5
- **Wie bewerten Sie die Stabilität des Tools?** 5/5
- **Wenn Sie Erfahrung mit anderen Codegenerierungstools haben, wie schneidet dieses Tool im Vergleich ab?** 4/5

7.7.3. Spezifisches Feedback

Einzelne Kommentare zum Tool beinhalten Vorschläge zur Verbesserung und persönliche Eindrücke zur Nutzung:

- Frage 11 (Verbesserungsvorschläge):
 - "Prettier wird nicht mit installiert, eventuell prüfen, ob vorhanden und dann Option anbieten."
 - "Generierung von API Calls in zusätzlichen Programmiersprachen wie PHP und Plain JavaScript."
- Frage 12 (Abschließender Gedanke):
 - "Das Tool erhöht meine Produktivität beim Erstellen von API Aufrufen stark."
 - "Ausbau der Bedienungsanleitung. Die Readme ist ausreichend, wenn man schon mal mit Azure DevOps gearbeitet hat. Das Tool an sich kann einem viel Zeit sparen, sofern die API-Spezifikation sauber vorliegt"

7.8. Abschließende Gedanken

Zusammenfassend lässt sich sagen, dass das Tool trotz seiner Einschränkungen und der kleinen Stichprobe gut angenommen wurde. Die vorgeschlagenen Verbesserungen zeigen jedoch, dass weitere Entwicklungen und Anpassungen notwendig sind, um den Bedürfnissen aller Benutzer gerecht zu werden. Zukünftige Änderungen könnten sich darauf konzentrieren, die Repräsentativität der Ergebnisse zu erhöhen und das Tool auf eine breitere Palette von Entwicklungsumgebungen auszuweiten.

7.9. Evaluation des theoretischen Ansatzes

7.9.1. Methodischer Ansatz

Der theoretische Ansatz dieser Arbeit bietet eine solide Grundlage für die praktische Anwendung der untersuchten Technologien und Methoden. Durch die Kombination aus theoretischen Überlegungen und praktischen Implementierungen konnte eine effektive Brücke zwischen Konzept und Anwendung geschlagen werden.

Jedoch könnte die theoretische Tiefe in bestimmten Bereichen der Arbeit stärker sein. Eine detailliertere Auseinandersetzung mit den theoretischen Modellen und Prinzipien hinter den verwendeten Technologien könnte die wissenschaftliche Strenge der Forschung verbessern. Zukünftige Arbeiten könnten davon profitieren, indem sie diese theoretischen Aspekte ausführlicher beleuchten, um ein noch umfassenderes Verständnis der Materie zu erlangen.

7.9.2. Evaluation der verwendeten Quellen

Die in dieser Arbeit verwendeten Quellen umfassen eine breite Mischung aus aktuellen und älteren Referenzen. Dies hilft dabei, ein umfassendes Verständnis der behandelten Technologien und ihrer Entwicklung zu gewährleisten. Die Verwendung älterer, etablierter Quellen ermöglichte es, die langfristige Relevanz und die historischen Grundlagen der Technologien zu unterstreichen, was für das tiefgreifende Verständnis der Materie unerlässlich ist.

Obwohl ältere Quellen wertvolle Einblicke in die historische Entwicklung bieten, besteht das Risiko, dass sie veraltete Informationen enthalten, die nicht mehr den aktuellen technologischen Standards entsprechen. In der vorliegenden Arbeit wurde darauf geachtet, solche Informationen kritisch zu bewerten und im Kontext neuerer Forschungen und Entwicklungen zu interpretieren. Diese methodische Herangehensweise stellt sicher, dass die verwendeten Quellen trotz ihres Alters relevante und korrekte Einblicke bieten und zur wissenschaftlichen Strenge der Arbeit beitragen.

8. Zusammenfassung und Ausblick

8.1. Erreichte Ergebnisse

Im Rahmen dieses Praktikums wurden bedeutende Fortschritte in der Entwicklung von optimierten Entwicklungsworkflows durch die Generierung von Code und den Einsatz von OpenAPI erzielt. Dies wurde insbesondere durch die Integration von Angular und .NET in eine effiziente und erweiterbare Codegenerierungsarchitektur realisiert. Die Implementierung des neuen Systems führte zu einer deutlichen Reduktion von Boilerplate-Code und optimierte die Entwicklungsprozesse erheblich. Durch die Automatisierung wurden wiederkehrende Aufgaben reduziert, wodurch sich die Entwicklungszeit verkürzte und die Produktivität innerhalb der Entwicklerteams gesteigert wurde.

8.2. Ausblick

8.2.1. Erweiterbarkeit der Ergebnisse

Die entwickelte Lösung zeichnet sich durch ihre modulare Architektur aus, die nicht nur die Integration zusätzlicher Funktionen innerhalb des Angular-Frameworks erleichtert, sondern auch eine Ausweitung auf andere Technologien ermöglicht. Dies könnte beispielsweise die direkte Dokumentation des Angular-Codes umfassen, erweiterbar durch die automatische Speicherprogrammierbare Steuerung (SPS)-Dokumentation, um die sich die Bachelorarbeit meines Kollegen Florian Merlau dreht.

Durch den Einsatz von LLMs und Natural language processing kann die Codegenerierung zudem direkt aus den dokumentierten Spezifikationen generiert werden, oder auch der generierte Code direkt je nach Bedarf um visuelle Komponenten erweitert werden.

8.2.2. Übertragbarkeit der Ergebnisse

Die Flexibilität der entwickelten Lösung erlaubt ihre Anwendung nicht nur innerhalb von Angular-Projekten, sondern auch die Adaptation und Erweiterung für andere Softwareentwicklungsumgebungen und -frameworks. Dies zeigt sich besonders in der Möglichkeit, die Grundprinzipien der Codegenerierung auf andere Systeme zu übertragen und dort ähnliche Optimierungen durchzuführen. Langfristig könnte dies auch branchenübergreifend auf andere Unternehmen übertragen werden. Die Anpassungsfähigkeit der Lösung bietet Chancen für eine branchenübergreifende Implementierung.

Literatur

- [1] *about npm*. 2024. URL: <https://docs.npmjs.com/about-npm> (besucht am 08.03.2024).
- [2] Thinktecture AG. *Blazor in .NET 7: Server, WebAssembly & Hybrid – ein Wegweiser*. Zugriff am 2024-03-07. 2024. URL: <https://www.thinktecture.com/contributions/blazor-in-net-7-server-webassembly-und-hybrid-ein-wegweiser/>.
- [3] Lisa Heneghan Albert Ellis. *Harvey Nash / KPMG Creative CIO survey 2016*. 2016. URL: <https://assets.kpmg.com/content/dam/kpmg/xx/pdf/2016/09/harvey-nash-kpmg-cio-survey-2016.pdf> (besucht am 13.03.2024).
- [4] Angular Team. *Angular Developer Survey 2023*. 2023. URL: <https://blog.angular.io/angular-developer-survey-2023-86372317c95f> (besucht am 30.03.2023).
- [5] Alexander Budzier Bent Flyvbjerg. *Why Your IT Project May Be Riskier Than You Think*. 2011. URL: <https://hbr.org/2011/09/why-your-it-project-may-be-riskier-than-you-think> (besucht am 13.03.2024).
- [6] David Hencke Bobbie Johnson. *Not fit for purpose: £2bn cost of government's IT blunders*. 2008. URL: <https://www.theguardian.com/technology/2008/jan/05/computing.egovernment> (besucht am 14.03.2024).
- [7] Mike Cohn. *Agile Estimating and Planning*. Page 235. Prentice Hall, 2007.
- [8] Redaktion ComputerWeekly.de. *Boilerplate (Boilerplate Code)*. n.d., last updated: 2021. URL: <https://www.computerweekly.com/de/definition/Boilerplate-Boilerplate-Code> (besucht am 18.03.2024).
- [9] John W. Creswell. *Research Design: Qualitative, Quantitative, and Mixed Methods Approaches*. 3. Edition. SAGE Publications Inc., 2009.
- [10] Alan M. Davis. *201 Principles of Software Development*. McGraw-Hill Inc, 1995.
- [11] Maurice Dawson u. a. *Integrating Software Assurance into the Software Development Life Cycle (SDLC)*. volume 3, number 6 page 49-53. 2010.
- [12] Ecma International. *ECMAScript® 2022 Language Specification*. Accessed: 2024-03-25. 2022. URL: <https://ecma-international.org/publications-and-standards/standards/ecma-262/>.

- [13] *Einführung in .NET*. Zugriff am 2024-03-07. 2024. URL: <https://learn.microsoft.com/de-de/training/modules/dotnet-introduction/>.
- [14] *Einführung in .NET Core*. Zugriff am 2024-03-07. 2024. URL: <https://learn.microsoft.com/de-de/dotnet/core/introduction>.
- [15] Neil Ernst. *A Field Study of Technical Debt*. 2024-03-18. 2015. URL: <https://insights.sei.cmu.edu/blog/a-field-study-of-technical-debt/>.
- [16] *ESLint: What, Why, When, How*. 2021. URL: <https://dev.to/shivambmgupta/eslint-what-why-when-how-5fld> (besucht am 11. 04. 2024).
- [17] Martin Fowler. *Technical Debt*. 2019. URL: <https://martinfowler.com/tags/technical%20debt.html#:~:text=Technical%20Debt%20is%20a%20metaphor,by%20Martin%20Fowler> (besucht am 24. 03. 2024).
- [18] Dave Gavigan. *The History of Angular*. 2018. URL: <https://medium.com/the-startup-lab-blog/the-history-of-angular-3e36f7e828c7> (besucht am 05. 03. 2024).
- [19] The Standish Group. *Standish Group 2015 CHAOS Report*. 2015. URL: https://www.standishgroup.com/chaosReport/index#myModal_42 (besucht am 13. 03. 2024).
- [20] Heitec AG *Firmenportrait*. 2023. URL: <https://www.heitec.de/de/ueber-uns/unternehmen/firmenportrait> (besucht am 05. 03. 2024).
- [21] IDC. *Developer Insights Report A Global Survey of Today's Developers*. 2015. URL: <https://developersalliance.org/research/reports/developer-insights-report-2015/> (besucht am 14. 03. 2024).
- [22] Andreas Johannsen. *Basiswissen für Software-Projektmanager im klassischen und agilen Umfeld*. 1. Auflage, Anne Kramer, Horst Kostal, Ewa Sadowicz. dpunkt.verlag, 2017.
- [23] Kasia Kowalska. *The great dilemma. Agile or waterfall?* 2019. URL: <https://www.boldare.com/blog/agile-vs-waterfall/> (besucht am 14. 03. 2024).
- [24] Lumen Learning. *Symbolic Interactionist Theory*. 2024. URL: <https://courses.lumenlearning.com/wm-introductiontosociology/chapter/reading-symbolic-interactionist-theory/> (besucht am 24. 03. 2024).
- [25] Kevin Ljung und Javier Gonzalez-Huerta. *To Clean-Code or Not To Clean-Code: A Survey among Practitioners*. arXiv:2208.07056v1 [cs.SE]. Blekinge Institute of Technology, 371 79, Karlskrona, Sweden. Aug. 2022.
- [26] Holger Schwichtenberg Manfred Steyer. *Moderne Webanwendungen mit ASP.NET MVC & Javascript*. 2. Auflage. O'Reilly, 2017.
- [27] Neil Ford Mark Richards. *Handbuch moderner Softwarearchitektur*. 1. Auflage. O'Reilly, 2021.
- [28] Robert C. Martin. *Clean Architecture*. Prentice Hall, 2017.

- [29] Robert C. Martin. *Clean Code: A Handbook of Agile Software Craftsmanship*. Prentice Hall, 2009.
- [30] Jürgen Laartz Michael Bloch Sven Blumberg. *Delivering large-scale IT projects on time, on budget, and on value*. 2012. URL: <https://www.mckinsey.com/capabilities/mckinsey-digital/our-insights/delivering-large-scale-it-projects-on-time-on-budget-and-on-value> (besucht am 13.03.2024).
- [31] Microsoft. *TypeScript - GitHub Repository*. Accessed: 2024-03-25. 2024. URL: <https://github.com/Microsoft/TypeScript>.
- [32] uvm. Mike Beedle Arie van Bennekum. *das agile Manifest*. Das agile Manifest wurde von vielen renommierten Softwareentwicklern und -theoretikern wie Kent Beck, Robert C. Martin, etc. unterschrieben. 2001. URL: <https://agilemanifesto.org/> (besucht am 09.03.2024).
- [33] Munich Business School. *Stakeholder - Einfach erklärt*. Available online: <https://www.munich-business-school.de/l/bwl-lexikon/stakeholder>. 2024. (Besucht am 24.03.2024).
- [34] Oracle. *Was versteht man unter Big Data?* 2024. URL: <https://www.oracle.com/de/big-data/what-is-big-data/> (besucht am 24.03.2024).
- [35] John Ousterhout. *A Philosophy of Software Design*. Yaknyam Press, 2018.
- [36] *Project Management Statistics: Trends and Common Mistakes in 2024*. 2024. URL: <https://teamstage.io/project-management-statistics/> (besucht am 14.03.2024).
- [37] Guenther Ruhe. *Optimization in Software Engineering: A Pragmatic Approach*. https://doi.org/10.1007/978-3-030-32489-6_9. 2020.
- [38] Scrum Alliance. *State of Scrum 2017-2018*. 2017. URL: [https://www.scrumalliance.org/ScrumRedesignDEVSite/media/ScrumAllianceMedia/Files%20and%20PDFs/State%20of%20Scrum/2017-SoSR-Final-Version-\(Pages\).pdf](https://www.scrumalliance.org/ScrumRedesignDEVSite/media/ScrumAllianceMedia/Files%20and%20PDFs/State%20of%20Scrum/2017-SoSR-Final-Version-(Pages).pdf) (besucht am 15.03.2024).
- [39] Vinit Sharma. *Angular Framework - From Its First Steps To Adulthood*. 2024. URL: <https://www.clariontech.com/blog/angular-framework-from-its-first-steps-to-adulthood> (besucht am 05.03.2024).
- [40] SmartBear Software. *OpenAPI Specification*. Zugriff auf die vollständige Open-API Spezifikation. 2023. URL: <https://swagger.io/specification/> (besucht am 10.04.2023).
- [41] Sebastian Springer. *node.js das umfassende handbuch*. 4. Auflage. Rheinwerk Verlag, 2016.
- [42] *Stackoverflow development surveys*. Stackoverflow surveys von 2018-2023. 2023. URL: <https://insights.stackoverflow.com/survey> (besucht am 30.03.2024).

- [43] Manfred Steyer. *Angular Das Praxisbuch zu Grundlagen und Best Practices*. 3. Auflage. O'Reilly, 2021.
- [44] Thamhain und Wilemon. „Criteria for controlling projects according to plan“. In: *Project Manager Journal* (1986).
- [45] TypeScript Team. *TypeScript - JavaScript For Any Scale*. Accessed: 2024-03-25. 2024. URL: <https://www.typescriptlang.org/>.
- [46] Inc. VersionOne. *15th Annual State of Agile™ Survey*. Sponsored by VersionOne Inc. Conducted by Analysis.Net Research. 2021.
- [47] Inc. VersionOne. *9th Annual State of Agile™ Survey*. Sponsored by VersionOne Inc. Conducted by Analysis.Net Research. 2015.
- [48] Stefan Wagner und Melanie Ruhe. *A Systematic Review of Productivity Factors in Software Development*. <https://arxiv.org/abs/1801.06475>. arXiv:1801.06475v1 [cs.SE]. 2018.
- [49] *What is Prettier?* 2023. URL: <https://prettier.io/docs/en/> (besucht am 11. 04. 2024).
- [50] Sofia Yermolaieva. *Communication Challenges in Agile Teams from The Communication Theory Perspective*. Conference Paper. Conference: At Innopolis University, Russia. DOI: 10.1145/3393822.3432327. 2020.

A. Anhang Interview

Allgemeine Zufriedenheit und Bewertung:

Frage: Wie zufrieden sind Sie mit dem aktuellen Workflow auf einer Skala von 1 bis 10?

Antwort: 3

Frage: Können Sie erklären, was zu dieser Bewertung geführt hat?

Antwort: Der Hauptgrund für diese Bewertung liegt in dem Fehlen eines klar definierten Workflows. Ohne eine deutliche Struktur oder Richtlinien ist es schwierig, Aufgaben effizient zu bewältigen oder den Erfolg zu bemessen.

Wahl von Workflow und Methoden:

Frage: Welche Tools nutzen sie um den Workflow effizient zu gestalten?

Antwort: Wir setzen ein Jira Kanban Board ein, um die Abläufe präzise auf die spezifischen Anforderungen jedes einzelnen Projekts zuzuschneiden. Dadurch können wir den Projektfortschritt verfolgen.

Frage: Wie beeinflusst diese Entscheidung Ihren täglichen Arbeitsablauf und die Teamdynamik?

Antwort: Das Jira Kanban Board hilft bei der Einhaltung von festgelegten Aufgabenpaketen, um eine gegenseitige Rücksichtnahme im Team zu unterstützen. Fullstack-Entwickler haben die Möglichkeit, ihre Aufgaben entsprechend ihrer Selbsteinschätzung flexibel zu gestalten. Das Kanban Board dient lediglich als eine flexible Richtlinie.

Stärken des aktuellen Workflows:

Frage: Welche Teile ihres aktuellen Workflows und der verwendeten Methode funktionieren gut? Bitte nennen Sie spezifische Beispiele.

Antwort: Ein Vorteil unseres aktuellen Workflows ist die hohe Flexibilität, die es ermöglicht, effektiv auf Änderungen zu reagieren. Diese Flexibilität zeigt sich darin, dass Anpassungen im Projektverlauf vorgenommen werden können.

Frage: Wie haben diese Aspekte konkret zur Produktivität und Effizienz Ihrer Projekte beigetragen?

Antwort: Die Flexibilität des Workflows ermöglicht es auch unkonventionelle Richtungen einzuschlagen und uns schnell an wechselnde Bedingungen anzupassen.

Verbesserungspotential:

Frage: Welche Aspekte Ihres aktuellen Workflows und der verwendeten Methode könnten verbessert werden? Bitte nennen Sie spezifische Beispiele.

Antwort: Obwohl unser Workflow und unsere Methoden grundsätzlich funktionieren, sehen wir Verbesserungspotential in der Planbarkeit und Effizienz unserer Prozesse.

Offenheit für neue Ansätze:

Frage: Wie offen sind Sie gegenüber der Einführung neuer Methoden im Workflow?

Antwort: Da unser Hauptanliegen darin besteht, stetige Verbesserungen und effizienzsteigernde Änderungen zu fördern, sind wir sehr offen dafür, neue Methoden in unseren Arbeitsablauf einzuführen.

Frage: Haben Sie bestimmte Bedenken oder Hoffnungen bezüglich neuer Verfahren oder Arbeitsabläufe?

Antwort: Unser Ziel bei der Implementierung neuer Prozesse oder Arbeitsmethoden ist es, unsere Abläufe effizienter und schneller zu machen. Allerdings gibt es Bedenken darüber, wie gut diese neuen Methoden bei den Angestellten ankommen und ob sie bereit sind, den veränderten Arbeitsablauf anzunehmen.

Erfahrungen mit alternativen Ansätzen:

Frage: Haben Sie Erfahrungen mit alternativen Ansätzen zur Software Entwicklung (z.B. Fullstack gegenüber spezialisierten Teams)? Was sind die Vor- und Nachteile im Vergleich, wenn ja?

Antwort: Wir verfügen über Erfahrungen mit spezialisierten Softwareentwicklungsteams sowie mit Fullstack-Konzepten, abhängig von den Projektanforderungen. Der Fullstack-Ansatz ermöglicht eine effiziente Lösung mit geringerem Bedarf an Koordination, erfordert jedoch ein größeres Kompetenzspektrum von Einzelpersonen. Obwohl spezialisierte Teams eine verbesserte Arbeitsteilung ermöglichen, erfordern sie mehr Kommunikation.

Frage: Können Sie bestimmte Beispiele geben, bei denen eine andere Arbeitsweise oder Methode besser oder weniger effektiv war als die aktuelle?

Antwort: Ein Fall, in dem ein anderer Ansatz wirksamer war, ist die Anwendung von Codegenerierung in Verbindung mit einem agilen Ansatz, der durch Sprints gekennzeichnet ist. Diese Methode hat sich als effektiv erwiesen, da sie im Gegensatz zu traditionelleren, starrerem Ansätzen eine schnelle Iteration und kontinuierliche Optimierung begünstigt.

Unterstützungsbedarf:

Frage: Welche Ressourcen oder Unterstützung benötigen Sie, um eine neue Methode oder Anpassungen im Workflow erfolgreich umzusetzen?

Antwort: Für eine erfolgreiche Umsetzung neuer Methoden oder Anpassungen im Workflow brauchen wir vor allem Werkzeuge, die den Entwicklungs-Prozess unterstützen und verbessern.

Tiefere Einsichten:

Frage: Können Sie mir sagen, welche Erfahrungen Sie gemacht haben, bei denen Ihr aktueller Workflow oder die Methode zum Projektmanagement besonders effektiv oder ineffektiv war?

Antwort: Der gegenwärtige Arbeitsablauf war ineffektiv, insbesondere wenn sich die Datenstrukturen innerhalb von Teams veränderten.

Frage: Wie beeinflusst Ihr aktueller Workflow Ihre Arbeit und Ihre Fähigkeit, auf unvorhergesehene Probleme zu reagieren?

Antwort: Auf unvorhergesehene Probleme zu reagieren, hat der aktuelle Workflow Auswirkungen, insbesondere aufgrund der Notwendigkeit von Anpassungen im Frontend, Backend, Datenbank und API. Die Teamkommunikation erschwert diese Anpassungen zusätzlich und resultiert in einer verminderten Fähigkeit, auf Probleme prompt zu reagieren.

Frage: Welche Änderungen am Arbeitsablauf oder an der Methode würden Ihrer Meinung nach den größten Einfluss auf die Projektarbeit haben?

Antwort: Die Implementierung einer Automatisierung für das Interface zwischen Client und Backend würde einen signifikanten Einfluss auf die Entwicklung haben, indem sie einen Arbeitsschritt eliminiert und das Anpassen von Datenstrukturen vereinfacht. Eine solche Neuerung hätte zur Folge, dass Änderungen nur noch an einer zentralen Stelle vorgenommen werden müssten, anstatt zahlreiche Ressourcen in Gang zu setzen. Diese Optimierung würde die Arbeitseffizienz deutlich steigern.

Frage: Können Sie ein Beispiel für eine solche Veränderung geben, die bereits umgesetzt wurde oder möglicherweise umgesetzt werden könnte?

Antwort: Bei einem Kundenprojekt wurde eine Modifikation implementiert, die die automatische Erstellung von Clientklassen ermöglichte. Nachdem eine neue Client-Klasse generiert und in das System eingefügt wurde, stand sie den Entwicklern zur Nutzung bereit, was den Arbeitsprozess erheblich beschleunigte.

B. Anhang Code

B.1. Vergleich Dateiaufbau

```
1 import { mergeMap as _observableMergeMap, catchError as
  _observableCatch } from 'rxjs/operators';
2 import { Observable, throwError as _observableThrow, of as
  _observableOf } from 'rxjs';
3 import { Injectable, Inject, Optional, InjectionToken } from
  '@angular/core';
4 import { HttpClient, HttpHeaders, HttpResponse, HttpResponseBase
  } from '@angular/common/http';
5 export const API_BASE_URL_SERVICE = new
  InjectionToken<string>('API_BASE_URL_SERVICE');
6
7 @Injectable()
8 export class ClientService { ... }
9
10 export class Order implements IOrder { ... }
11
12 export interface IOrder { ... }
13
14 export class ProblemDetails implements IProblemDetails { ... }
15
16 export interface IProblemDetails { ... }
17
18 export class Product implements IProduct { ... }
19
20 export interface IProduct { ... }
21
22 export class ApiException extends Error { ... }
23
24 function throwException(message: string, status: number,
  response: string, headers: { [key: string]: any; }, result?:
  any): Observable<any> { ... }
25
26 function blobToText(blob: any): Observable<string> { ... }
```

Quelltext B.1: Aufbau einer Datei im bekannten CodeGeneration Tool

```

1  /*This file was generated automatically.
2  *****Your Imports may be
   different.*****/
3  import { Injectable } from '@angular/core';
4  import { HttpClient } from '@angular/common/http';
5  import { Observable } from 'rxjs';
6  import { ENVIRONMENT_BASE_API_URL } from
   './environments';/*Don't forget to change the path and add
   a env variable with a connection string.*/
7  import { catchError, map } from 'rxjs/operators';
8  import { handleError } from './ChangeFilePath'/*Don't forget to
   change the path of the handleError file.*/;
9  import { Order, ProblemDetails, Product } from
   './interfaces/ChangeFilePath';/*Don't forget to change the
   path of the interfaces file.*/
10
11
12  @Injectable({
13     providedIn: 'root'
14  })
15  export class ProductService {
16     constructor(private http: HttpClient) { }
17
18     public getProduct(): Observable<Array<Product>> {...}
19
20     public postProduct(body: Product): Observable<Product> {...}
21
22     public getId(id: string): Observable<Product> {...}
23  }

```

Quelltext B.2: Aufbau einer Datei im neuen CodeGeneration Tool

B.2. Vergleich ErrorHandling

```
1 function throwError(message: string, status: number,  
   response: string, headers: { [key: string]: any; }, result?:  
   any): Observable<any> {  
2   if (result !== null && result !== undefined)  
3     return _observableThrow(result);  
4   else  
5     return _observableThrow(new ApiException(message, status,  
       response, headers, null));  
6 }
```

Quelltext B.3: Errorhandling im bekannten CodeGeneration Tool

```
1 /*This file was generated automatically.*/  
2 import { HttpResponse } from '@angular/common/http'  
3 import { throwError } from 'rxjs'  
4  
5 export function handleError(error: HttpResponse) {  
6   let errorMessage = 'An unknown error has occurred.'  
7   if (error.error instanceof ErrorEvent) {  
8     // Client-side or network error occurred.  
9     errorMessage = `An error occurred: ${error.error.message}`  
10  } else {  
11    // The backend returned an unsuccessful response code.  
12    errorMessage = `Server returned code ${error.status}, error  
      message is: ${error.message}`  
13  }  
14  console.error(errorMessage)  
15  // Return an observable with a user-facing error message.  
16  return throwError(() => new Error(errorMessage))  
17 }
```

Quelltext B.4: Errorhandling im neuen CodeGeneration Tool

C. Anhang Umfrage

C.1. Umfragebogen

Bewertung:	1	2	3	4	5	
Ich bin gar nicht zufrieden	☐	☐	☐	☐	☐	Ich bin sehr zufrieden
Das Tool ist schwer zu bedienen/erlernen	☐	☐	☐	☐	☐	Das Tool ist leicht zu bedienen/erlernen
Das Design und die Benutzeroberfläche sind nicht ansprechend	☐	☐	☐	☐	☐	Das Design und die Benutzeroberfläche sind sehr ansprechend
Das Tool erfüllt nicht meine Erwartungen	☐	☐	☐	☐	☐	Das Tool übertrifft meine Erwartungen
Das Tool spart mir keine Zeit	☐	☐	☐	☐	☐	Das Tool hilft mir erheblich, Zeit zu sparen
Die Performance ist sehr langsam	☐	☐	☐	☐	☐	Die Performance ist sehr schnell
Ich stoße sehr häufig auf Leistungsprobleme	☐	☐	☐	☐	☐	Ich habe keine Leistungsprobleme
Fehler treten sehr häufig auf	☐	☐	☐	☐	☐	Fehler treten selten auf
Das Tool ist sehr instabil	☐	☐	☐	☐	☐	Das Tool ist sehr stabil
Dieses Tool ist schlechter als andere, die ich verwendet habe	☐	☐	☐	☐	☐	Dieses Tool ist besser als andere, die ich verwendet habe

C.2. Fragen

1. **Verbesserungsvorschläge:** Welche Funktionen oder Verbesserungen würden Sie sich für das Tool wünschen?
2. **Abschließender Gedanke:** Welche weiteren Kommentare oder Vorschläge haben Sie zum Tool?